UNIX capable RISC-V SoC based on the VirtIO framework

(RISC-V SoC oparty na infrastrukturze VirtIO obsługujący systemy z rodziny UNIX)

Michał Błaszczyk

Praca magisterska

Promotorzy: dr Marek Materzok

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

29 stycznia 2026

Abstract

In this thesis, I present the design of a complete general-purpose computer developed from the ground up. The primary objective is to provide a straightforward educational example of a computer system that deliberately trades performance for simplicity. To reduce complexity, the system adopts the VirtIO-MMIO interface for peripheral devices and incorporates a small dedicated processor that implements the VirtIO protocol in software. Developing such a system requires substantial effort, including the design of numerous hardware components as well as the preparation of a complete software stack. The outcome of this work is a fully functional general-purpose computer capable of running Doom on Linux, both in simulation and on an FPGA board.

W niniejszej pracy przedstawiam projekt kompletnego komputera ogólnego przeznaczenia opracowanego od podstaw. Głównym celem jest dostarczenie przejrzystego, edukacyjnego przykładu systemu komputerowego, który świadomie przedkłada prostotę nad wydajność. Kluczowym pomysłem umożliwiającym redukcję złożoności jest zastosowanie interfejsu VirtIO-MMIO dla urządzeń peryferyjnych oraz wykorzystanie niewielkiego dedykowanego procesora implementującego protokół VirtIO w warstwie programowej. Opracowanie takiego systemu wymaga znaczących nakładów pracy, zarówno przy projektowaniu licznych komponentów sprzętowych, jak i przy przygotowaniu kompletnego stosu oprogramowania. Rezultatem pracy jest w pełni funkcjonalny komputer ogólnego przeznaczenia, zdolny do uruchomienia gry Doom w systemie Linux – zarówno w symulacji, jak i na płytce FPGA.

To my mom

Contents

Chapter 1

Introduction

The reach and impact of general-purpose computers are enormous. Billions of such systems are widely employed in smartphones, PCs, gaming consoles, modern cars, robots, and many other embedded devices.

While computers become more and more ubiquitous, interest in understanding the inner workings of such systems is also growing. With the arrival of FPGA boards and the introduction of open ISAs, many open-source hardware designs implementing complete general-purpose computers have been introduced. Such systems range from single-core pipelined CPUs connected to basic I/O peripherals via Wishbone bus to complex multi-core out-of-order CPUs attached to several peripheral devices via PCI bus. Although such designs provide a great educational resource for anyone who wants to learn how to build a functional computer, the majority of those systems consist of tens of thousands of lines of source code and require a considerable time commitment to understand them.

VirtIO [34] RISC-V SoC (VVSoC) is a general-purpose computer design that trades speed for simplicity and utilizes the VirtIO framework to simplify the implementation of peripheral devices. Although the main goal of the system is to provide a straightforward educational example of an FPGA computer design, it is fully capable of running UNIX-like operating systems and even some sophisticated applications such as Doom.

1.1 What Is a General-Purpose Computer?

A general-purpose computer is a system that can carry out a wide range of computing tasks. It is advanced enough to run an operating system that controls the devices available on the platform and provides an execution environment for user applications. A typical general-purpose computer contains the following components:

- CPU – a processing unit equipped with mechanisms required by an operating

system to implement the abstractions provided to user processes (e.g., virtual address spaces). In the case of systems with additional dedicated processors, the CPU is also referred to as the application processor.

- Main memory – a random-access memory device directly accessible by the CPU. It stores program data and instructions, paging structures, and I/O buffers.
- Peripheral devices – a set of several devices that usually includes a console, keyboard, display, and an external storage device.
- Bus infrastructure – a bus system that connects all the components together and enables them to communicate.

1.2 The Design Approach

There are many possible approaches one can take when it comes to designing a general-purpose computer targeted at an FPGA board. The two main options are as follows:

- CPU – these types of designs concentrate on constructing the central processing unit, which contains an implementation of a bus master. Subsequently, a system builder is used to complement the processor with main memory and selected peripheral devices and connect them using the preferred type of bus.
- CPU + devices + bus infrastructure – this is a ground-up approach. In these designs, the entire system is implemented by the designer, and no system builder is used.

The most popular system builders include Platform Designer [23] (for Altera/Intel FPGAs), IP Integrator [37] (for Xilinx/AMD FPGAs), and LiteX SoC builder [11] (an open-source alternative). Although system builders are extremely valuable and provide a portable way to construct complex systems, there are also some downsides to using them:

- They can significantly increase the code size of a design.
- The generated source code can be hard to understand.
- The features implemented by the provided components can significantly exceed the basic functionality needed for a simple general-purpose computer to operate.

VVSoC focuses on providing an implementation of an entire computer system except for the low-level memory controller, which is provided by the vendor of the target board.

1.3 Peripheral Devices Made Easy

One of the most challenging aspects of developing a general-purpose computer is implementing peripheral devices. Some devices may utilize advanced techniques such as Direct Memory Access (DMA), which entail significant code space and resources to implement.

When designing a peripheral device, we need to choose an interface that the device will implement and provide to the operating system. There are two conventional options:

- Well-known interface – this approach chooses an interface that is already known to the target operating system, and an appropriate device driver is available. On the downside, the interface of choice may be difficult to implement in hardware and may substantially increase the code size of the system.
- Custom interface – in such a case, we can define our own interface that concentrates on simplifying the hardware implementation. The drawback is that we need to provide a device driver so the operating system can handle the device.

VVSoC takes another innovative approach introduced by the creators of RV-SoC [18]. A well-known interface is chosen, and the system is augmented with a simple processor dedicated to implementing the interface in software. This is the crucial idea behind the design, which drastically simplifies peripheral device implementation.

1.3.1 Chosen Interface

Since the dedicated processor effectively acts as a hypervisor by implementing peripheral devices, we have two main choices when selecting a well-known interface: device emulation and paravirtualization.

Device emulation has the following characteristics:

- The hypervisor intercepts every access to the device and emulates the behavior of real hardware.
- To the guest operating system, the emulated device is indistinguishable from the corresponding hardware component, allowing the use of standard, unmodified drivers.
- Emulation can significantly reduce performance due to frequent hypervisor interceptions and the need to reproduce complex hardware operations faithfully.

On the other hand, paravirtualization has the following characteristics:

- The device does not attempt to mimic real hardware.
- The device interface and the communication protocol between the device and its driver are defined by a straightforward, well-specified standard.
- The guest operating system must provide specialized drivers.
- Paravirtualization offers considerable performance gains for the following reasons:
  - Hypervisor interceptions are far less frequent.
  - The communication protocol is simpler.
  - Shared memory can be used for direct data transfers.

VVSoC chooses paravirtualization over device emulation. Taking inspiration from RVSoC, VirtIO-MMIO [34, Sec. 4.2] has been selected as the target interface and is applied to all supported peripheral devices. Such an approach has the following advantages:

- VirtIO is a standard interface for virtual devices supported by many UNIX-like operating systems, including Linux [15].
- VirtIO supports multiple device classes through a single transport and negotiation model.
- Each VirtIO-MMIO device requires only a handful of 32-bit registers for configuration and queue control.
- The VirtIO specification is publicly documented and open source.
- Since UNIX and QEMU [2] support VirtIO-MMIO, a platform similar to VVSoC can be easily emulated on a standard PC, which facilitates software development, especially porting new operating systems.

The processor dedicated to implementing VirtIO devices is referred to as the VirtIO core.

1.4 Processor Architecture

Another fundamental characteristic of a computer system is the architecture implemented by the CPU. VVSoC employs RISC-V [32, 33] as the processor architecture for both application and VirtIO processing units. There are several reasons for this choice:

- RISC-V is an open architecture (unlike ARM and x86), which means that anyone can design a core utilizing this architecture without paying any royalties.

- It is a load-store architecture with a small and modular instruction set. Modularity makes it possible to design both simple controllers and sophisticated processors capable of running advanced operating systems.

- RISC-V has a substantial developer and user community. It is supported by GCC, LLVM, and major operating systems, including Linux.

- The architecture is widely used in computer design and architecture courses all around the world, which makes it a perfect choice for an educational general-purpose computer system.

Both VVSoC processors use the little-endian byte ordering.

1.5 Word Length

The word length of a processor determines the size of integer registers, the maximum size of a memory transaction, and the maximum size of an address space. To support a modern operating system, we can choose between the 32-bit and 64-bit variants of the target architecture.

Selecting a 64-bit word length has the main benefits given below:

- 64-bit registers allow native 64-bit integer arithmetic.

- The bigger word size increases memory throughput.

- Some operating systems may support only the 64-bit variant of the architecture.

On the other hand, choosing the 32-bit variant has the following main advantages:

- The smaller word size decreases the complexity of the hardware. This enables implementation on FPGA boards with fewer resources and facilitates inspection of the results of synthesis and implementation.

- Reducing pointer size results in smaller memory usage, enabling efficient RAM and cache utilization.

Both VVSoC processors implement the 32-bit variant of the RISC-V architecture.

1.6 Hardware Description Language

One of the most important technological choices is selecting the Hardware Description Language (HDL) used to write the source code of the project. HDLs can be divided into two fundamental groups based on the design methodology:

- High-Level Synthesis (HLS) languages,
- Register-Transfer Level (RTL) languages.

High-level synthesis languages have the following characteristics:

- HLS enables designing hardware using software-style code.
- The developer designs at a higher abstraction, by describing the algorithm and letting the compilation tool decide how to create the hardware structure.
- Development of algorithmic designs is much faster with high-level synthesis.
- Due to the higher abstraction, the developer has significantly less control over the exact hardware produced.
- Tools supporting high-level synthesis vary in quality.

In contrast, register-transfer level languages have the characteristics given below:

- The developer describes the hardware directly at the level of individual registers.
- RTL provides full control over timing, resource usage, pipelines, and architecture.
- RTL designs are more predictable in terms of performance and area.
- Designing with register-transfer level languages requires more time.

Since high-level synthesis languages cannot express low-level structural hardware, VVSoC was designed using a register-transfer level language.

1.6.1 Chosen RTL Language

VVSoC has been developed using the SystemVerilog RTL language, which has the following benefits:

- SystemVerilog is widely used and well established in FPGA/ASIC design and verification. It is supported by all major tools, including:

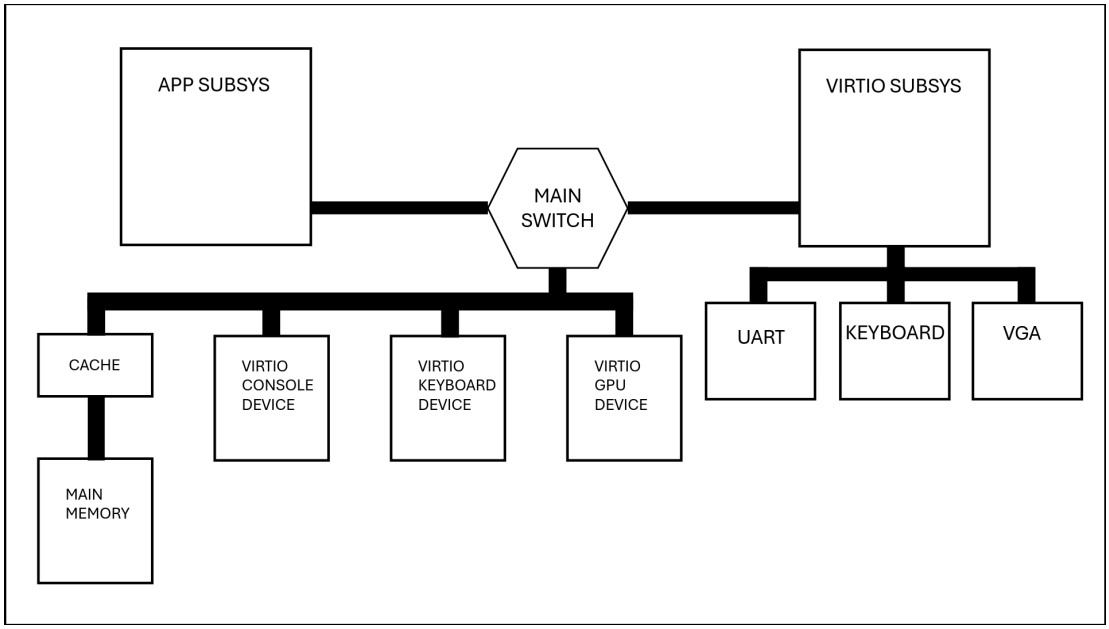

Figure 1.1: System organization

  - FPGA toolchains,
  - ASIC toolchains,
  - Simulators,
  - Formal verification tools,
  - Linting/static-analysis tools.

- Choosing SystemVerilog often simplifies integration with third-party IP, since a large portion of modern and legacy IP cores are written in SystemVerilog or Verilog, and SystemVerilog is fully compatible with Verilog.
- The language provides useful abstractions (structs, enums, interfaces, packages, loops) while keeping the register-transfer layer methodology.
- Because SystemVerilog is extensively used in digital logic and hardware design education globally, RTL designs implemented in this language tend to be more readily understood by students, facilitating learning and comprehension.

1.7 System Organization

VVSoC is organized into three main parts, as shown in Figure 1.1:

- Application subsystem – this component contains the application CPU, which is the main processor of the system, and its purpose is to execute the operating system code. The remaining elements composing the subsystem support the operation of the CPU (e.g., a flash controller).

- Global elements – this is a set of devices accessible to both subsystems. The key element is the main memory, which is complemented by a physically indexed cache. Furthermore, for each supported VirtIO device, there is a hardware implementation of the register map defined by the VirtIO-MMIO specification. At present, VVSoC implements three VirtIO devices: console, keyboard, and display.
- VirtIO subsystem – this part contains the secondary CPU responsible for implementing the VirtIO devices. This processor is not available to the system programmer and should be considered an implementation detail. The VirtIO subsystem is the only component that performs actual operations on peripheral devices.

The two processors operate in parallel and do not communicate directly with each other. Each core implements a bus master and can access components located in the global space.

1.8 Parametrization

VVSoC has been designed with configurability in mind. Therefore, instead of using hard-coded constants in the HDL, SystemVerilog parameters are employed. The configurable parameters can be divided into two groups:

- Parameters that do not depend on the target board (e.g., the number of cache sets).
- Board-specific parameters (e.g., the size of main memory).

Board-independent parameters are defined in the `param` SystemVerilog package, located at `hw/include/param.svh` within the VVSoC repository [39], whereas board-specific ones are included in the `board` package as explained in 9.2. All configurable parameters are described in this thesis alongside the corresponding hardware components.

Chapter 2

Application Subsystem

The application subsystem is composed of the following components, as shown in Figure 2.1:

- Application core – the central processing unit of the system. This is where an operating system runs.
- SiFive Core-Local Interruptor [31] (CLINT) – this device implements privileged memory-mapped timer registers defined by the RISC-V specification.
- Boot memory – a simple read-write memory used by the app core during the boot process.
- Debug console – a primitive console device. It is capable of transmitting characters through the serial transmitter and does not require any additional setup.
- Flash controller – this module allows the app core to read bytes stored in the flash memory.

2.1 Application Core

The goal of VVSoC is to be a functional general-purpose computer suited for running a UNIX-like operating system. To achieve this ambition, we need a CPU that provides the means for the operating system to implement basic abstractions and features. Since we have chosen RISC-V as the target architecture, this entails implementing a particular set of Instruction Set Architecture (ISA) extensions.

2.1.1 RISC-V Extensions for Running an OS

Standard UNIX-like operating systems require the following minimal set of RISC-V extensions to be supported:

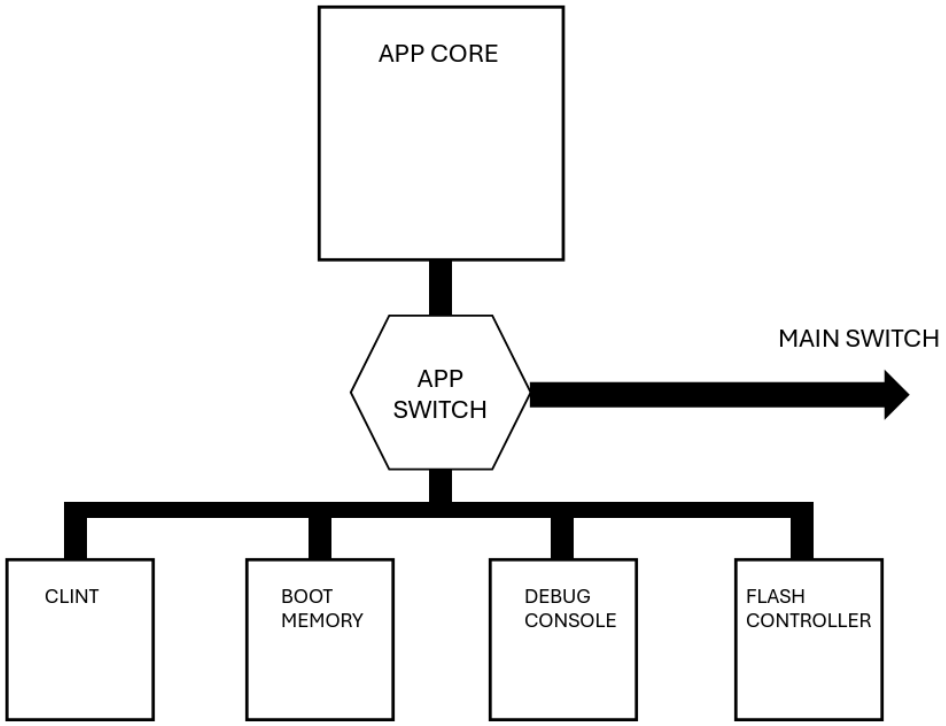

Figure 2.1: Application subsystem

- Base integer instruction set (RV32I) – defines a basic set of integer registers, integer computational instructions, control transfer instructions, load and store instructions, memory ordering instructions, an environment call instruction, and a breakpoint instruction.
- Integer multiplication and division (M) – a set of two integer multiplication and two integer division instructions.
- Atomic instructions (A) – defines a load-reserved instruction, a store-conditional instruction, and a set of read-modify-write instructions.
- Instruction-fetch fence (Zifencei) – an instruction cache flushing operation.
- Control and status instructions (Zicsr) – a set of instructions that operate on the Control and Status Registers (CSRs).

2.1.2 RISC-V Privilege Modes

RISC-V defines the following three modes of operation:

- Machine-mode (M-mode) – this is the highest privilege mode with unconstrained access to the hardware platform. In a standard system, a runtime firmware operates in this mode and provides services to the operating system.

- Supervisor-mode (S-mode) – this is the mode in which an operating system executes. The S-mode controls paging and provides an execution environment for user applications.
- User-mode (U-mode) – this is the lowest privilege mode. It is intended for user processes.

Operating systems require all these modes of operation, along with the corresponding control and status registers, as well as privileged instructions, to be implemented.

2.1.3 RISC-V Control and Status Registers

Control and status registers are special-purpose registers defined by the RISC-V specification. CSRs have three main functions:

- Control of the processor's operation – this includes enabling, disabling, and delegating interrupts.
- Monitoring the processor's state – this involves keeping track of the current privilege level, reporting the cause of an exception, and maintaining performance counters.
- Facilitating system-level functions – this includes traps and virtual memory.

The length of a CSR matches the native register size (in our case, 32 bits wide). For each CSR, there is a minimum privilege level that is required to access the register; for instance, registers starting with the letter 'm' are only accessible when the processor operates in the machine mode.

2.1.4 The Design Approach

Processor design can be classified into three main approaches:

- Single-cycle processor – it is a processor that carries out a single instruction within a single clock cycle. This method is only suitable for very simple processing units. The main disadvantages include a lengthy critical path and the requirement of separate instruction and data memories.
- Multi-cycle processor – in such a core, instruction execution is divided into several one-cycle stages. The main downside is that only one instruction is executed at a time, which means that most states are left idle.
- Pipelined processor – this approach extends the multi-cycle solution by enabling each stage to be utilized by a different instruction at the same time.

Although introducing a pipeline can greatly increase the performance, it makes the design substantially more complex and can considerably increase the code size. Pipelined processors become even more sophisticated when they support all privilege modes.

VVSoC values simplicity over performance; therefore, the application core is a multicycle processor.

2.1.5 Implementing Read-Modify-Write Instructions

RMW instructions can be implemented in software. In such an approach, whenever an RMW instruction is encountered, an illegal instruction is raised, and the machine-mode runtime firmware has to emulate the instruction. This method has the following two downsides:

- Implementing read-modify-write instructions in software is significantly slower than a hardware implementation.
- Some runtime firmwares (including OpenSBI [19]) require the atomic instructions extension to be implemented entirely in hardware.

VVSoC implements read-modify-write instructions in hardware. For this purpose, two additional stages have been introduced:

- The second execution stage – executed after the first memory access stage. It contains an ALU dedicated to performing the modification stage of RMW instructions.
- The second memory access stage – carried out after the second execution stage. This is where the modified data is written back to memory.

2.1.6 Interrupt Controller

In contrast to sophisticated multi-core systems, which utilize a Platform-Level Interrupt Controller [27] (PLIC), VVSoC routes all interrupts directly to a Hart-Level Interrupt Controller [26] (HLIC).

VVSoC has the following interrupt sources:

- Machine timer,
- VirtIO console [34, Sec. 5.3] (routed to platform interrupt 0),
- VirtIO GPU [34, Sec. 5.7] (routed to platform interrupt 1),
- VirtIO keyboard [34, Sec. 5.8] (routed to platform interrupt 2).

2.1.7 Memory Management Unit

The 32-bit variant of the RISC-V architecture defines a single address-translation scheme called Sv32 [33, Sec. 4.3]. The page size is set to 4096 bytes, and each page table contains 1024 entries, which implies that the paging structure has two levels.

The Memory Management Unit (MMU) is responsible for translating virtual addresses to physical addresses. The MMU module implemented in VVSoC is also responsible for performing memory operations, both for accessing paging structures during hardware page walks and for executing loads and stores on the processor's behalf.

Currently, VVSoC does not support any writable external storage devices, and the dirty flag is not fully supported.

Translation Lookaside Buffer

The Translation Lookaside Buffer (TLB) is a cache for Page Table Entries (PTE). In case of a TLB hit, the virtual page number can be translated into the corresponding physical page number without any additional accesses to the paging structure.

The TLB employed by the application core has the following organization:

- 4-way set-associative.
- 4 sets.
- Address Space Identifier (ASID) support.
- Round-robin replacement policy.

The set-associative TLB defines the following configurable parameters:

- `TLB_SETCNT` – number of sets.
- `TLB_LINECNT` – number of lines per set.

In addition to the default TLB organization described above, VVSoC contains an optional direct-mapped TLB. Only the `TLB_SETCNT` parameter applies to this TLB variant.

Instruction Cache

To accelerate the instruction fetching stage, an instruction cache (ICACHE) has been introduced. The primary purpose of the cache is to speed up the execution

of consecutive instructions and small loops. The ICACHE holds a single line of the global main memory cache, which is a contiguous block of 16 instructions.

The cache is virtually indexed and virtually tagged. Although this approach makes the design simpler, it requires flushing under the following circumstances:

- Privilege level change.
- Modification of the `SATP` register.
- TLB flushing instruction.
- Instruction-fetch fence instruction.

It should be noted that designing an instruction cache is significantly less challenging than designing a data cache. Since VVSoC does not support Physical Memory Attributes [33, Sec. 3.6] (PMA), each memory region is considered cacheable, including the segments shared between the two cores. Therefore, a data cache would require careful synchronization with main memory operations performed by the VirtIO core. For this reason, a data cache is currently not implemented.

2.2 CLINT

The RISC-V Advanced Core Local Interruptor (ACLINT) specification [25] defines the MTIMER memory-mapped device that provides timer functionality to RISC-V cores. The MTIMER device defines two 64-bit machine-level registers:

- `MTIME` – a fixed-frequency monotonic counter.
- `MTIMECMP` – a time compare register.

Whenever the value of `MTIME` becomes greater than or equal to `MTIMECMP`, a machine timer interrupt is activated.

The SiFive core-local interruptor device is a widely used implementation of the MTIMER device. The advantage of choosing CLINT as the timer capability provider is that most operating systems supporting RISC-V contain appropriate device drivers to handle it.

2.3 Boot Memory

The application core cannot start fetching instructions from main memory immediately after power-up. Although the initial calibration of DRAM may be handled in hardware, main memory is volatile, so the required binary images must be loaded into it first.

The boot memory is a read-write Block RAM (BRAM) device that stores the binary image of the bootloader, which is executed by the application core during the boot process. Block RAM is a special type of on-chip memory on an FPGA board that can be initialized with the data provided during the FPGA configuration process. Although this approach is simple, it has some drawbacks, which are investigated further in 6.1.1.

The boot memory defines a single configurable parameter `BMEMSZ` that specifies the size of the memory in bytes.

2.4 Debug Console

The debug console provides the functionality of writing a single character at a time to the serial transmitter. Although the VirtIO console device is available, there are several advantages to including a simple debug console device:

- The debug console can be used immediately after power-up, with no further setup needed. In particular, no main memory operations are necessary.
- It does not rely on the VirtIO core to execute a character transmission request.
- It can be used by runtime firmware after the boot process has completed. In the case of the VirtIO console, after the operating system takes control over the device, it is no longer available to the M-mode firmware.

2.5 Flash Controller

We need a non-volatile storage device to store images that will be loaded into main memory by the application core. VVSoC chooses flash memory for this purpose, which has the following benefits:

- Most FPGA boards capable of implementing a general-purpose computer contain a flash memory device.
- Flash controllers are relatively simple to design, especially since we only require a reading operation.
- With a suitable flash controller in place, flash accesses do not require any additional configuration.

Although the implementation of a flash controller depends on the target FPGA board, for example, the flash chip may be connected via SPI, or a NOR flash interface may be used, VVSoC defines a common interface that each implementation must provide, namely a bus slave.

The flash controller defines a single board-specific configurable parameter `FLSZ` that specifies the size of the flash memory in bytes.

Chapter 3

VirtIO

Virtual Input/Output (VirtIO) is a standard that defines a device interface and device-driver communication protocol for a broad range of devices (console, disk, network, etc.), designed for paravirtualized devices implemented by a hypervisor.

Although the VirtIO device interface and communication protocol were originally designed for virtualized environments where the device is typically implemented by a hypervisor, the same standard can also be used in real systems, where the device is implemented directly in hardware.

VVSoC uses VirtIO-MMIO as the interface for all peripheral devices and avoids the complexity of implementing the protocol in hardware by delegating it to a dedicated processor (called VirtIO core) that handles the VirtIO protocol in software.

3.1 Communication Protocol

Communication between a VirtIO driver and device occurs through shared memory and interrupts. Each device exposes one or more data structures called virtual queues (virtqueues). VirtIO defines two types of virtqueues: split and packed. In VVSoC, only the conventional split virtqueues are used; therefore, the term virtqueue in the remainder of this thesis refers specifically to the split variant.

Each virtqueue consists of three shared areas:

- Descriptor table (also called the descriptor area),
- Available ring (also called the driver area),
- Used ring (also called the device area).

The device and driver negotiate the size of each virtqueue during the boot process as described in 3.2.2. This single value defines the number of entries in all three components of a virtqueue.

3.1.1 Descriptor Table

The descriptor table is an array of buffer descriptors. The table itself is written only by the driver; however, some of the buffers referenced by those descriptors may be written by the device. The layout of a virtqueue descriptor is shown below (`le` stands for little-endian):

```
struct virtq_desc {
    le64 addr;
    le32 len;
    le16 flags;
    le16 next;
};
```

The following is a description of each member of the structure:

- `addr` – guest physical address of the buffer.
- `len` – length of the buffer in bytes.
- `flags` – a set of flags indicating descriptor properties, such as whether the device is only allowed to read from or write to the buffer and whether this descriptor is part of a chained sequence.
- `next` – index of the next descriptor in the chain (valid only if the chaining flag is set).

Each descriptor is either read-only or write-only from the device's perspective. Drivers can chain multiple descriptors using the `next` field and the chaining flag. If there are read and write descriptors in the same chain, writable descriptors must come after all read-only descriptors.

3.1.2 Available Ring

The available ring is a cyclic array in which the driver places the indices of descriptors that are ready to be processed by the device. The available ring is written exclusively by the driver and read by the device. The layout of the available ring is given below:

```
struct virtq_avail {
    le16 flags;
    le16 idx;
    le16 ring[ /* Virtqueue size */ ];
};
```

The members of the available ring structure are described as follows:

- `flags` — properties of the available ring; unused in VVSoC.
- `idx` — monotonically increasing index of the next available slot in the ring. The value is not stored modulo the queue size; modulo arithmetic is applied only when accessing the array.
- `ring` — array of indices into the descriptor table. If a descriptor chain is used, only the index of the first descriptor in the chain is written to the ring.

When the driver wants to submit a request to the device, it performs the following steps:

1. Allocate as many descriptors from the descriptor table as needed for the request. If there are not enough descriptors available, abort the operation.
2. Allocate an entry in the available ring. If the ring is full, abort the operation.
3. Allocate a buffer in guest physical memory for each descriptor, fill the buffers that the device will read, and chain the descriptors if necessary.
4. Add the index of the first descriptor in the chain to the available ring.
5. Increment the `idx` field of the available ring.
6. Send a notification to the device indicating that new entries are available for processing. Sending notifications is described in 3.2.2.

The device maintains the last seen value of the `idx` field so it can determine which available entries are new.

3.1.3 Used Ring

The used ring is a cyclic array that the device employs to return processed buffers back to the driver. The used ring is written exclusively by the device and read by the driver. The layout of the used ring is similar to that of the available ring and is shown below:

```
1  struct virtq_used_elem {
2      le32 id;
3      le32 len;
4  };
5
6  struct virtq_used {
7      le16 flags;
8      le16 idx;
9      struct virtq_used_elem ring[ /* Virtqueue size */];
10 };
```

The `flags` and `idx` fields have meanings analogous to the corresponding fields in the available ring. `ring` is an array of pairs, each containing the following two fields:

- `id` – the index into the descriptor table. This value corresponds to the descriptor index originally placed into the available ring for the processed request.
- `len` – the total number of bytes written by the device. If the request used a descriptor chain, the data may be spread across multiple buffers.

When the device is ready to process a new available entry, it performs the steps below:

1. Read the descriptor index from the available ring and traverse the descriptor chain, processing each descriptor in order. If no chaining is used, only one descriptor is processed.
2. Perform the required I/O operation (for example, a disk device may issue a physical read or write request).
3. Write output data into buffers marked as writable.
4. Add an entry to the used ring containing the index of the first descriptor in the processed chain (the same index obtained from the available ring) and the total number of bytes written into writable buffers.
5. Increment the `idx` field of the used ring.
6. Generate an interrupt to notify the driver that new entries are available in the used ring.

The driver maintains the last seen value of `idx` so it can determine which used entries are new. When an interrupt arrives, the driver clears the pending interrupt by acknowledging it as described in 3.2.2, and performs the following actions for each new entry in the used ring:

1. Process the data received in writable buffers and hand it to other kernel layers.
2. Release each associated descriptor.
3. Release the corresponding available ring entry.

3.2 Device Interface

Although VirtIO defines a single logical device interface, the exact form of the interface depends on the selected transport option.

3.2.1 Transport Options

There are two main VirtIO transport options:

- VirtIO over PCI – the most popular transport option, with some of its main advantages being auto-discovery and high performance. The main disadvantage is the complexity of this solution, as the target system must provide support for the PCI bus.
- VirtIO over MMIO – an alternative that offers significantly lower complexity, at the cost of lacking auto-discovery and providing lower performance.

Since VVSoC values simplicity over complexity, the PCI bus is not implemented, and the VirtIO over MMIO transport option is used.

3.2.2 VirtIO-MMIO Register Map

Each VirtIO-MMIO device exposes a set of memory-mapped control registers, some of which are described below. The register names follow the VirtIO 1.3 specification [34, Sec. 4.2.2]:

- `DeviceID` – identifies the type of the VirtIO device (e.g., block device, network device).
- `QueueSel` – selects which virtqueue is affected by the following registers:
  - `QueueSizeMax` – maximum supported number of entries in the queue (read-only).
  - `QueueSize` – the actual number of entries the driver configures for the queue. This determines the sizes of the descriptor table, available ring, and used ring.
  - `QueueReady` – writing 1 to this register marks the queue as enabled.
  - `QueueDescLow` – lower and upper 32 bits of the guest-physical address of the descriptor table.
  - `QueueDriverLow` – lower and upper 32 bits of the guest-physical address of the available ring.
  - `QueueDeviceLow` – lower and upper 32 bits of the guest-physical address of the used ring.
- `Status` – current device status.
- `QueueNotify` – the driver writes a queue index to this register to notify the device that new buffers are available.

- `InterruptStatus` – bit 0 is set by the device when at least one of the enabled queues contains a new entry in the used ring.
- `InterruptACK` – the driver writes to this register to clear the pending interrupt.

The mandatory set of registers described above is followed by an optional, device-specific configuration space.

Chapter 4

Global Elements

The global space contains devices that are shared between the two subsystems. Main memory stores virtqueue areas and I/O buffers, whereas VirtIO devices provide an interface — implemented by the VirtIO subsystem — to the application core.

4.1 Cache

In order to speed up memory accesses, VVSoC is equipped with a cache with the following organization:

- Physically indexed and physically tagged.
- 4-way set-associative.
- 16 sets.
- 64-byte data block.
- Write-back strategy for a write-hit.
- Write-allocate strategy for a write-miss.
- Round-robin replacement policy.
- Implemented using block RAM.

The main motivation for choosing a physically indexed cache is to remove the need for synchronization between the subsystems.

The cache defines two configurable parameters:

- `CACHE_SETCNT` – number of sets.
- `CACHE_LINELEN` – number of bits in a single line.

From the processor's perspective, cache writes usually take only a single cycle, after which the processor can continue execution while the cache finalizes the write in the background. A stall occurs only if the processor issues a new request while the previous write is still being processed.

4.2 Main Memory

Despite the ground-up approach of implementing every system component, VVSoC makes an exception for the low-level memory controller. Designing a DRAM controller is a daunting task for a number of reasons:

- DRAM chips require precise timing control for every operation (activate, precharge, read, write, refresh).
- DRAM requires command sequencing, bank management, and address mapping.
- A controller must periodically schedule refresh commands without disrupting normal read/write operations.
- Small mistakes in timing or protocol handling can cause rare, hard-to-reproduce errors.

VVSoC employs a vendor-provided DRAM controller. The supplied IP provides a well-defined user interface that the main memory module relies on.

Main memory defines the following board-specific configurable parameters:

- `MMEMSZ` – size of the memory in bytes.
- `MMEM_DATALEN` – size of a single memory transaction in bits.

4.3 VirtIO Devices

VVSoC provides the following VirtIO devices:

- VirtIO console device,
- VirtIO keyboard device,
- VirtIO GPU device.

Although the implemented devices do not include any custom features beyond those defined by the VirtIO specification, they have been designed to support only

the basic functionality required by a standard UNIX-like operating system. Therefore, some of the less common services defined by the specification may be omitted.

Since VVSoC implements VirtIO devices in software using the VirtIO core, and actual peripheral device operations are performed in the VirtIO subsystem, the hardware implementation of each VirtIO device can be limited to the basic register map defined by the VirtIO-MMIO specification (3.2.2).

Each implemented VirtIO device defines a single configurable parameter:

- `VCD_QUEUESIZEMAX` for the console device.
- `VKD_QUEUESIZEMAX` for the keyboard device.
- `VGD_QUEUESIZEMAX` for the GPU device.

The parameter specifies the value returned by the `QueueSizeMax` VirtIO-MMIO register for each virtqueue for the given device. The `QueueSizeMax` register is explained in 3.2.2.

4.3.1 VirtIO Console Device

Providing a console device is very valuable, as it offers a command-line interface to the operating system running on the target platform. Moreover, the console is the primary way the operating system communicates early boot messages.

VVSoC uses the most basic variant of the console device, where only a single port is provided. The port contains a pair of virtqueues:

- receiveq – queue for receiving bytes.
- transmitq – queue for sending bytes.

In the basic variant, the VirtIO console does not provide a configuration space.

Reading

The following is a typical read communication scenario:

1. For each entry in the available ring in the receive queue, the OS prepares a separate I/O buffer and notifies the device.
2. Whenever the UART input FIFO in the VirtIO subsystem contains some bytes, and there are pending entries in the receive queue, the VirtIO core transfers the data from the FIFO to the I/O buffers, adds entries to the used ring, and asserts an interrupt.

3. The OS handles the interrupt, processes the received data, and reuses the released entries.

Writing

The list below presents a typical writing communication:

1. Whenever there are bytes to be sent to the console, and there is an available entry in the transmit queue, the OS allocates an I/O buffer, copies the output data to the buffer, attaches the buffer to an available ring entry, and sends a notification to the device.
2. The VirtIO core reads the characters from the buffer and transfers them one by one to the UART device. After all characters have been sent, an entry is added to the used ring, and an interrupt is raised.
3. The OS receives the interrupt and registers an available descriptor as well as an available transmit entry.

4.3.2 VirtIO Keyboard Device

The main advantage of providing a keyboard device is that, when paired with a display device, it enables the user to interact with the system directly, without relying on a console connected to a PC.

The keyboard device is an instance of the VirtIO input device, which is intended for human interface devices. VirtIO input devices generate events defined by the evdev [13, Sec. 1.3.1.1] (event device) Linux kernel layer.

The configuration space is used to further identify the input device and report the set of supported events. The keyboard device included in VVSoC supports the following two kinds of events [13, Sec. 2.1]:

- `EV_KEY` – a key has been pressed or released.
- `EV_SYN` – a synchronization event that reports the arrival of `EV_KEY` events.

In the current implementation, the configuration space is not supported, and the appropriate information is hardcoded in the device driver for the VirtIO input device.

The keyboard device contains two virtqueues:

- eventq – this queue is used to send input events to the OS.
- statusq – this queue is used to send status updates (e.g., LED updates) from the OS to the device.

Currently, the VirtIO keyboard device provided by VVSoC supports only the event queue.

Reading

The following is a standard communication scenario:

1. For each entry in the available ring in the event queue, the OS allocates an evdev event structure and notifies the device.
2. Whenever the keyboard event FIFO in the VirtIO subsystem is non-empty, and there are pending entries in the event queue, the VirtIO core copies the events from the FIFO to the designated evdev structures. Afterwards, the core adds a synchronization event, inserts entries into the used ring, and asserts an interrupt.
3. The OS handles the interrupt, passes the events to the evdev layer, and reuses the released queue descriptors and available ring entries.

4.3.3 VirtIO GPU Device

Providing a display device is essential for enabling the user to run graphical applications on the target platform.

The graphics adapter device contains two virtqueues:

- controlq – queue for sending control commands.
- cursorq – queue for sending cursor updates.

In the current implementation, VVSoC supports only the control queue.

The configuration space for the VirtIO GPU device provided in VVSoC contains one notable register called `num_scanouts`. It specifies the maximum supported number of displays and is fixed to one.

Command Protocol

The VirtIO GPU device defines a protocol in which each message consists of two parts: a request and a response. Consequently, each command is represented as a descriptor chain, with the request placed in the initial portion of the chain and the response placed in the remaining portion. In most commands implemented in VVSoC, both the request and the response fit into a single descriptor.

Communication

A standard communication scenario is given below:

1. The OS issues the `GET_DISPLAY_INFO` command.
2. The OS creates a 2D resource on the device's side using the `RESOURCE_CREATE_2D` command.
3. The OS allocates main memory for a framebuffer.
4. The OS attaches the framebuffer as backing storage for the previously created resource with the `RESOURCE_ATTACH_BACKING` command.
5. At this point, the framebuffer is set up and ready. The subsequent processing continues in the following loop:
   (a) The OS modifies the framebuffer in RAM.
   (b) The OS triggers an update of the resource with the `TRANSFER_TO_HOST_2D` command.
   (c) The VirtIO core copies the framebuffer from main memory to the hardware framebuffer in the VirtIO subsystem.
   (d) The OS issues the `RESOURCE_FLUSH` command.

In the case of all the above-mentioned commands, the OS notifies the device, the VirtIO core provides a suitable response and raises an interrupt, and finally, the OS handles the interrupt.

The implemented VirtIO GPU device has been specifically designed to support the aforementioned communication scheme. Using a significantly different setup may not be supported.

4.4 Arbitration

The application core and the VirtIO core operate in parallel; therefore, they may wish to access the same resource at the same time. For this reason, it is necessary to provide arbitration to avoid race conditions. The following priority rules apply:

- The application core has higher priority for main memory accesses. Practice indicates that this approach yields modest performance improvements.
- The VirtIO core has higher priority for VirtIO device accesses. The rationale behind this choice is to provide the application core with the most recent state of the devices.

Chapter 5

VirtIO Subsystem

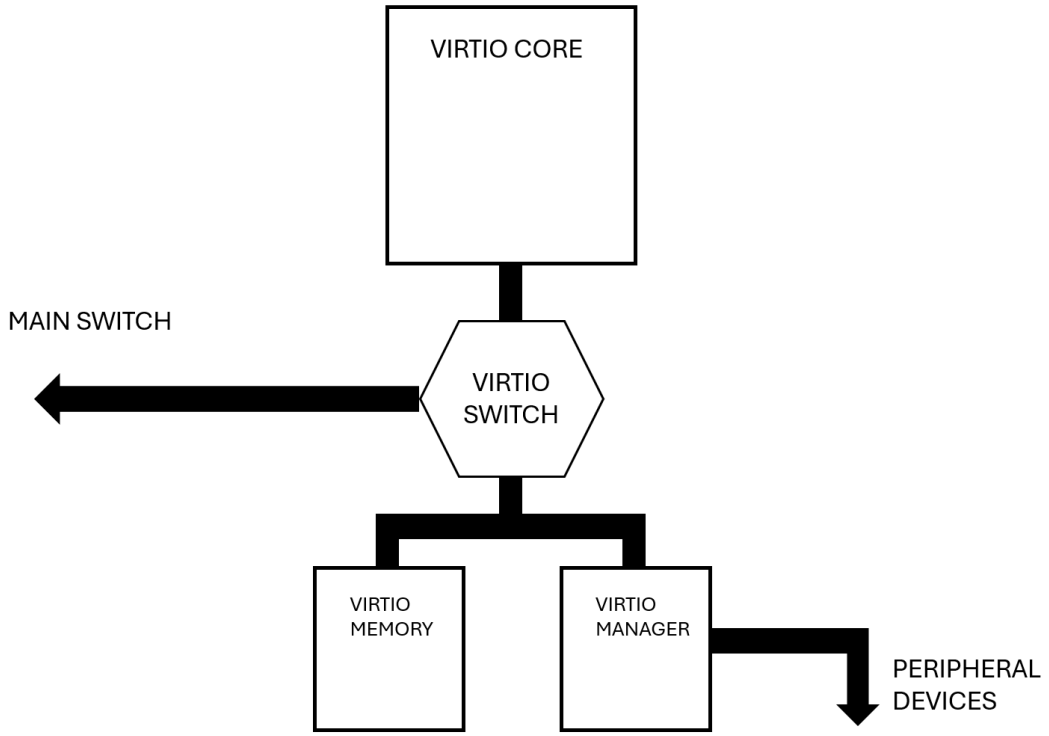

Figure 5.1: VirtIO subsystem

The VirtIO subsystem is organized into the following components, as shown in Figure 5.1:

- VirtIO core – a simple processor dedicated to implementing VirtIO devices.
- VirtIO memory – a read-write memory used by the VirtIO core to store program code and data.
- VirtIO manager – this module manages the operation of the entire VirtIO subsystem. Every peripheral device is directly connected to this component.

5.1 VirtIO Core

The VirtIO core executes a lightweight firmware that handles requests sent by the operating system through virtqueues. Because the core executes only a single, highly trusted firmware, this allows us to make the following simplifications in the design of the core:

- We only have to implement the ISA extensions required by the firmware; therefore, the only supported extension is the base integer instruction set (RV32I).
- Since the code is considered secure and the core operates directly on physical addresses, only the machine privilege level needs to be supported.
- Since the trusted code is assumed not to cause any exceptions, and there are no interrupt sources routed to the VirtIO core, the trap support can be removed altogether.

As a result, the VirtIO core is a straightforward design of a minimal RISC-V processor.

5.2 VirtIO Memory

The VirtIO memory is a read-write block RAM device similar to the boot memory that stores the binary image of the program executed by the VirtIO core. The main memory is used by the VirtIO core exclusively for accessing virtqueues and I/O buffers. All remaining loads and stores are performed on the VirtIO memory.

The VirtIO memory defines a single configurable parameter `VMEMSZ` that specifies the size of the memory in bytes.

5.3 VirtIO Manager

The VirtIO manager is the most sophisticated component of the VirtIO subsystem. It is responsible for:

- Maintaining notifications for each virtqueue.
- Instructing the VirtIO core to handle a specific virtqueue.
- Informing VirtIO devices that a virtqueue request has been successfully handled.
- Providing the VirtIO core with a memory-mapped interface to perform basic I/O operations.
- Controlling peripheral devices.

5.3.1 Communication with the VirtIO Core

Although the VirtIO core can directly access main memory and VirtIO devices, it relies on the VirtIO manager to perform actual operations on peripheral devices. The manager implements the following set of memory-mapped registers accessible to the core:

- `REQ` – receive a request to handle. A read from this register will stall until there is a pending virtqueue request to process. The returned data indicates which device and virtqueue to handle.
- `UART_RX` – read a single byte from the UART input FIFO.
- `UART_TX` – send a single byte to the UART. If the UART transmitter is already busy, this operation will stall.
- `KBD` – read a single keyboard event from the input FIFO.
- `VGA_UPDATE` – set the color of the pixel at the current index in the hardware framebuffer and advance the index to the next pixel.
- `FINISH` – a write to this register informs the manager that the core has finished handling a request. The written value indicates whether the processing was successful (if it was, an interrupt will be asserted).

The VirtIO core operates in the following loop:

1. Find out which virtqueue to handle by reading the `REQ` register.
2. Process as many entries in the virtqueue as possible.
3. Report to the VirtIO manager that the processing has finished using the `FINISH` register.

5.3.2 Peripheral Devices

VVSoC implements three peripheral devices as shown in Figure 5.2:

- UART – used to implement the VirtIO and debug console devices.
- Keyboard – used to implement the VirtIO keyboard device.
- VGA – used to implement the VirtIO GPU device.

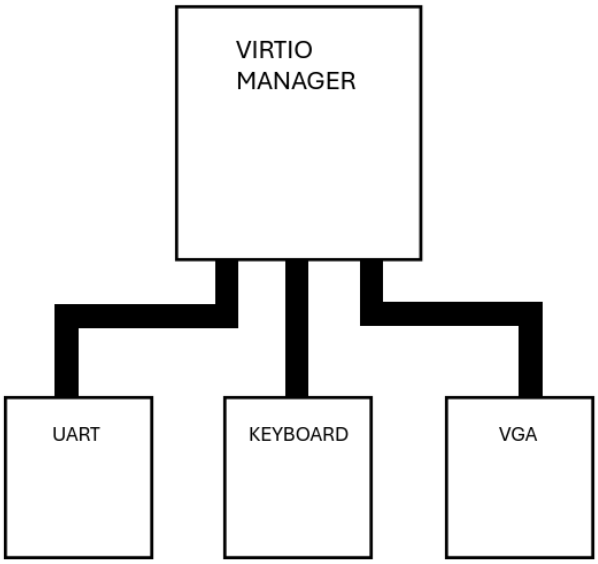

Figure 5.2: VirtIO manager

UART

VVSoC chooses the Universal Asynchronous Receiver/Transmitter (UART) hardware communication protocol to implement the console device. There are three primary causes for relying on UART:

- Most FPGA boards are equipped with a USB-to-UART bridge, which enables serial communication with the connected PC.
- UART controllers are straightforward to implement and require only modest hardware resources.
- Mainline operating systems, such as Linux and Windows, provide drivers for USB serial devices.

The implemented UART controller has the following characteristics:

- 8-bit words.
- 2 stop bits.
- No parity.
- Baud rate of 115200.

The VirtIO manager maintains an input FIFO for received bytes. The FIFO entries are subsequently read by the VirtIO core using the `UART_RX` register while handling a read request sent by the OS to the VirtIO console device.

Currently, no output queue is implemented; therefore, whenever the VirtIO core issues a write through the `UART_TX` register, the manager initiates a transfer in the UART controller, and a subsequent transmit request must wait until the transfer is completed.

The UART controller defines a single configurable parameter `UART_BAUD_RATE`.

Keyboard

In contrast to the console and GPU devices, VVSoC does not rely on a single protocol to implement the keyboard device; for instance, the keyboard could be connected via PS/2 or USB. Instead, the VirtIO manager utilizes a common interface that each implementation is expected to provide.

Since VirtIO input devices transfer data to the OS in the form of evdev events, the evdev event structure has been selected as the common interface. Consequently, a PS/2 keyboard controller must convert received scan codes to evdev events before passing them to the manager.

The VirtIO manager maintains a FIFO for input events sent by the keyboard controller. The VirtIO core receives the events through the `KBD` register and sends them further to the OS via virtqueues.

In the current implementation, writing to the keyboard device is not supported.

VGA

VVSoC employs the Video Graphics Array (VGA) graphics protocol to implement the GPU device, which has the following benefits:

- Although more and more modern boards rely on digital interfaces such as HDMI, VGA connectors are commonly included in the majority of older and education-focused FPGA boards.
- A basic VGA controller is a lightweight module that only has to generate horizontal and vertical synchronization signals along with RGB color outputs.
- VGA's fixed pixel clock and sync timing are well documented and easy to reproduce.
- Most modern monitors still support VGA input, especially in educational environments.

The VirtIO manager maintains a hardware framebuffer implemented with block RAM, which the VirtIO core updates pixel by pixel using the `VGA_UPDATE` register

whenever the OS issues the `TRANSFER_TO_HOST_2D` command to the VirtIO GPU device.

Although the standard VGA resolution is 640x480 pixels, the hardware framebuffer stores frames of size 320x240 pixels, which are scaled up by the VGA controller to produce images of the standard resolution. This approach has two main advantages:

- It significantly decreases block RAM usage.
- It speeds up frame updates by both the operating system (memory framebuffer) and the VirtIO core (hardware framebuffer).

The VGA controller defines a single board-specific configurable parameter `VGA_COLORLEN` that specifies the number of bits per color.

5.3.3 A Standard Scenario

To clarify the system components and their interactions, a standard operational scenario is described below:

1. The OS configures a VirtIO device, sets up virtqueues, and sends a notification to the device indicating that there are new requests to handle.
2. The VirtIO manager receives the notification and marks the virtqueue as ready for processing.
3. If the virtqueue can be handled (e.g., the appropriate input FIFO is non-empty), the manager decides to process the queue, and the `REQ` register read in the VirtIO core returns indicating the device and virtqueue to handle.
4. The VirtIO core processes the requests using the register map provided by the manager to access peripheral devices.
5. The VirtIO core writes to the `FINISH` register to report to the manager that requests have been handled successfully.
6. The VirtIO manager flags the virtqueue as not pending (unless new notifications have arrived in the meantime), and informs the VirtIO device that queue entries have been processed.
7. The VirtIO device asserts an interrupt.
8. The OS receives, acknowledges, and handles the interrupt.

5.3.4 Configurable Parameters

The VirtIO manager defines the following configurable parameters:

- `VMGR_UART_RX_FIFOSZ` – number of entries in the UART input FIFO.
- `VMGR_KBD_FIFOSZ` – number of entries in the keyboard input FIFO.

Chapter 6

Software Stack

Whereas the VirtIO core executes a fixed firmware at all times, the code executed by the application core can be structured in a hierarchy of four levels:

1. Primary bootloader – this is the first code executed upon power-up. It loads images into the main memory and prepares the execution environment for the next stage of the boot process.

2. Secondary bootloader/runtime firmware – executed from main memory. During boot, it performs further initialization and proceeds to the OS entry point. Moreover, this layer serves as the runtime firmware, providing the execution environment for the OS.

3. Operating system kernel – upon first entry, it initializes different kernel layers and performs device discovery. Eventually, it starts loading user-space programs. During regular execution, it provides services to user processes.

4. User processes – this is where user applications are executed. User-space programs rely on the abstractions provided by the OS and request services via system calls.

6.1 Primary Bootloader

Before the application core can start fetching instructions from main memory, the appropriate binary images must be copied from flash to DRAM. The software responsible for carrying out these operations is called the primary bootloader.

The source code for the bootloader can be found at the `fw/bootloader` directory within the VVSoC repository.

6.1.1 The Design Approach

There are several setups to consider when it comes to storing and executing a primary bootloader:

- BRAM – this is the simplest approach. In this setup, the primary bootloader is written to a block RAM device during FPGA configuration. The BRAM is also used by the bootloader for data and the stack. This approach has the following downsides:
  - Relying on the initialization of BRAM during FPGA configuration makes the design unsuitable for ASIC implementations.
  - Directly changing the primary bootloader requires recompiling the design.
  - Since the memory is writable, the images loaded from flash may corrupt the code stored in BRAM during execution.
- BROM + BRAM – this approach improves on the previous one by separating the single BRAM device into read-only and read-write memories. The primary bootloader is stored in the BROM, while the BRAM is used for data storage and the stack.
- BROM + BRAM + flash – in contrast to the second approach, the BROM stores a lightweight program that loads the primary bootloader from flash into the BRAM device. Although this setup still requires BRAM initialization, the primary bootloader can be updated simply by overwriting the flash image.
- Flash – in this approach, the primary bootloader is stored and executed directly from flash. Additionally, the flash is used for data storage and the stack. While this setup is the most portable, since it does not rely on block RAM initialization, it is also the most complicated, requiring flash writes to be implemented.

In the current implementation, VVSoC adopts the first approach, using a single BRAM device referred to as the boot memory.

6.1.2 Loaded Images

The primary bootloader loads the following images from the flash to the main memory:

- Device Tree Blob [6] (DTB) – description of the target platform.
- Secondary bootloader/runtime firmware – next stage of execution.
- Kernel – binary image of the target OS kernel.
- Initrd – initial RAM disk, which will be used as the root filesystem.

6.1.3 Further Operation

Once all images have been copied to main memory, the application core proceeds to fetch instructions from main memory, executing the secondary bootloader. The primary bootloader provides the following initial environment for the second stage:

- The core operates at the machine-level privilege mode.
- The first argument indicates the ID of the current processor. Since VVSoC has only one application core, the ID is set to zero.
- The second argument contains the physical address of the loaded DTB.
- CSRs are in the reset state; in particular, all interrupts are disabled.

Once the core has finished executing the primary bootloader, this code segment is never re-entered at any point.

6.2 Secondary Bootloader/Runtime Firmware

A typical RISC-V system includes an M-mode runtime firmware that serves the following purposes:

- It provides a standard execution environment for an operating system executing in S-mode. The firmware implements a set of services defined by the RISC-V Supervisor Binary Interface (SBI) specification [28], which the OS can invoke using the environment call instruction.
- The firmware handles platform-specific implementations of standard devices, such as a timer or a console.
- It performs the second stage of the boot process, including:
  - Setting up M-mode trap handlers.
  - Delegating specific interrupts and exceptions to S-mode.
  - Platform-specific initialization.

The main advantage of introducing a well-defined runtime firmware is that it provides operating systems with a consistent, hardware-independent interface, even across different target platforms.

6.2.1 OpenSBI

Although a few alternative solutions exist, OpenSBI is the standard SBI implementation used when building runtime firmware for a RISC-V target platform. Selecting OpenSBI offers several advantages:

- OpenSBI is the reference implementation of the SBI specification.
- It is open-source, licensed under BSD-2-Clause.
- OpenSBI implements a comprehensive set of SBI extensions, including both mandatory and optional ones.
- OpenSBI is actively maintained under the RISC-V ecosystem.
- It supports a wide range of RISC-V platforms, from QEMU virtual systems to physical boards.
- OpenSBI can be easily ported to a new platform by defining minimal platform operations.

VVSoC employs OpenSBI as the M-mode runtime firmware that implements the SBI specification.

6.2.2 OpenSBI on VVSoC

In order to port OpenSBI, a new target platform must be introduced. VVSoC defines a minimal platform that performs the following notable actions:

- Register and initialize an ACLINT MTIMER device.
- Define a lightweight debug console driver and register a console device.
- Delegate VirtIO console, keyboard, and GPU interrupts to S-mode.
- Transfer control to the operating system by jumping to a platform-specific entry point address.

Source

The source code for the OpenSBI VVSoC platform can be found on the `vvsoc` branch of the following fork of the OpenSBI repository: `https://github.com/MichalBlk/opensbi`.

6.2.3 Further Operation

Once the second stage of the boot process has completed, the core starts executing the OS kernel, with the initial state described below:

- The current privilege level is the supervisor mode.
- The arguments passed are the same as in the case of the primary bootloader.

- Memory translation is disabled.
- Interrupts are disabled.

Once OpenSBI has finished the initialization, and the control proceeds to the OS kernel, the runtime firmware will be re-entered in the following circumstances:

- A machine-mode timer interrupt is asserted.
- The operating system requests a service using an SBI call.
- An undelegated exception occurs (e.g., an illegal instruction exception).

6.3 Operating System Kernel

For an operating system to run on VVSoC, the kernel must meet two requirements:

- The 32-bit variant of the RISC-V architecture must be supported.
- The kernel must provide drivers for VirtIO-MMIO devices (at least for the VirtIO console).

To date, two UNIX-like operating system kernels have successfully run on VVSoC: Linux and Mimiker [17].

6.3.1 Device tree

In case of systems that do not include discoverable hardware, the operating system must be provided with a description of the device tree of the target hardware platform. The most common format of a device tree description is a Device Tree Source (DTS) with the corresponding compiled format called Device Tree Blob (DTB). DTS includes the following information:

- Kernel command line.
- Physical address and size of the initial RAM disk.
- CPU description, including:
  - Timer frequency.
  - ISA extensions.
  - Memory translation mode used by the operating system.
  - Local interrupt controller description.
- Physical memory boundaries.

- Physical address of the range reserved for the runtime firmware.
- Description of every provided device, including:
  - Physical boundaries of the MMIO range.
  - Interrupt routing.

DTS files for Linux are located in `fw/dts/linux`, whereas DTS files for Mimiker are located in `fw/dts/mimiker`.

6.3.2 Linux

There are several reasons why Linux is the primary choice for VVSoC:

- The kernel includes extensive RISC-V support.
- Linux provides drivers for all standard VirtIO devices and supports the MMIO transport option.
- Linux is completely open-source (GPL), with an enormous developer community.
- The kernel is modular and can be easily customized for minimal embedded setups.
- Linux supports standard POSIX APIs, which means that most existing applications, libraries, and tools will run unmodified.
- Linux is a proven foundation for stable and long-running embedded systems.
- Since Linux is widely studied in academia, it is an excellent choice for an educational project.

6.3.3 Linux on VVSoC

Since Linux has been ported to both variants of the RISC-V architecture (RV32 and RV64) and provides drivers for standard VirtIO devices, the kernel requires little to no changes to run on VVSoC. Only two minor modifications to the source code have been introduced:

- Since VVSoC does not implement the configuration space for the VirtIO keyboard device, the VirtIO input device driver has been altered to include the appropriate values without performing any configuration space accesses.
- Since VVSoC does not include a platform-level interrupt controller and relies solely on the hart-level interrupt controller, the VirtIO device interrupt registering code has been modified not to rely on a shared interrupt but instead to use a per-CPU interrupt.

Configuration

Over time, the kernel has continuously evolved, introducing numerous new features. As a result of these enhancements and improvements, the size of the kernel has grown significantly. The default kernel configuration produces a binary image that can exceed the flash memory capacity of FPGA-based SoCs. Therefore, the kernel size must be reduced for our purposes.

Linux is highly customizable, allowing most major kernel components to be compiled as loadable modules or omitted entirely if they are not required. One of the standard ways to configure the kernel is through the menuconfig tool. Although it provides a wide range of options that can considerably decrease the kernel's size, finding a minimal configuration manually can be tedious and error-prone.

VVSoC relies on the `tinyconfig` configuration target to produce a minimal kernel image, which offers several benefits:

- The resulting kernel is often only a few hundred kilobytes in size.
- The kernel compilation process becomes extremely fast.
- With most subsystems disabled, the boot time becomes substantially reduced.
- The configuration provided by tinyconfig is minimal, allowing us to incrementally enable only the features we need.

The following notable additional features have been enabled to complete the configuration:

- Initial RAM disk support.
- Enable `printk`.
- Support for ELF binaries.
- Support for shell scripts.
- Enable UNIX domain sockets.
- Enable TTY.
- Enable the generic input layer.
- Enable evdev.
- Platform bus driver for VirtIO-MMIO devices.
- VirtIO console driver.
- VirtIO input driver.

- Enable Direct Rendering Manager (DRM).
- VirtIO GPU driver.

The described configuration has been added as a new target called `vvsoc_defconfig`.

Source

The modified source code and the described configuration are available in the `https://github.com/MichalBlk/linux-vvsoc` repository.

6.3.4 Mimiker

Mimiker is an open-source UNIX-like operating system designed for education and research. It is developed by students at the University of Wrocław under the guidance of Krystian Bacławski. The project's main objective is to learn operating system design by studying existing systems (mainly from the BSD family) and reimplementing or adapting selected components for Mimiker according to the following principles:

- Minimalism,
- Simplicity,
- Code readability.

Mimiker has been chosen as the second target OS to run on VVSoC for two reasons:

- It originated at, and continues to be developed at, the University of Wrocław.
- Although I have ported Mimiker to RISC-V as part of my Bachelor's thesis [3], it has never successfully run on a physical RISC-V platform, only on an emulated one. Designing VVSoC helped me to identify and fix previously omitted kernel bugs. As a result, Mimiker can finally run on actual RISC-V hardware.

6.3.5 Mimiker on VVSoC

Although Mimiker supports both variants of the RISC-V architecture, it currently does not provide support for any VirtIO devices, unlike Linux. Since the current release of Mimiker does not support many user-space applications that utilize the keyboard or display, only a driver for the VirtIO console device has been implemented.

In addition to providing a driver for the VirtIO console, the following notable changes have been introduced:

- Since Mimiker assumes that the target platform includes a platform-level interrupt controller, the root device (rootdev) has been modified to support systems that rely only on the hart-level interrupt controller.
- Machine-dependent kernel virtual address space allocation has been fixed.
- A race condition in the callout module has been fixed.
- SBI timer updates have been fixed.
- GCC optimization level has been set to 2.

Source

The modified source code can be found on the `vvsoc` branch of the following fork of the Mimiker repository: `https://github.com/MichalBlk/mimiker`.

6.4 User Processes

In general, we can distinguish two components of the user-space:

- System libraries – for instance, the standard C library. The libraries can be either static or dynamic, and are linked against user programs.
- User programs – these include user applications, shells, as well as user-level daemons and services.

All these components are contained in a filesystem referred to as the root filesystem. On VVSoC, regardless of the target operating system kernel, the root filesystem is provided as an Initial RAM Disk (initrd).

6.4.1 Initial RAM Disk

The initial RAM disk is a temporary root filesystem contained entirely in RAM. It is typically used by operating systems during the startup process and contains a minimal set of components required to load additional kernel modules and mount the real root filesystem.

In many embedded systems without external storage devices, the initrd is used as the final root filesystem. This approach offers several benefits:

- Since the root filesystem is entirely in RAM, there is no delay in accessing slow storage devices.
- The kernel does not have to provide drivers for any storage devices.

- The kernel does not have to include support for complex filesystems, as only initrd support is required.
- Updating the root filesystem becomes extremely easy (no need for complex partitioning).
- The final initrd image stored in flash can be compressed.

Although relying on initrd as the final root filesystem brings significant simplifications, there are a few major drawbacks:

- Initrd occupies a substantial amount of precious operating memory that could otherwise be used by user applications.
- The boot process is considerably longer since the primary bootloader must copy the entire initrd from flash to RAM.
- Since RAM is volatile, any changes made during runtime are lost after a reboot.
- Because the final initrd image resides in flash memory, the overall size and number of user-space components must be minimized.

VVSoC currently supports only a single external storage device: flash memory. Since flash capacity is limited, write operations are not implemented, and the configuration is simpler with a RAM-based root filesystem, the root filesystem is not stored on flash. Instead, an initial RAM disk is used.

Depending on the selected operating system kernel, a different method is used to build the initrd image. In both cases, the final filesystem image is a cpio archive.

6.4.2 Linux

There are many tools that can provide a filesystem with a complete Linux user-space aimed at embedded systems. One of the most prominent is Buildroot [4].

Buildroot

Relying on Buildroot to generate a Linux root filesystem has the following advantages:

- Buildroot is designed to be straightforward to use.
- A wide range of processor architectures is supported, including RISC-V.
- Buildroot is optimized to produce minimal root filesystems, which is perfect for memory- and storage-constrained embedded devices.

- The tool is customizable and flexible. We can easily select libraries, packages, and features to include.
- Buildroot can build a cross-compilation toolchain tailored to our target system.

VVSoC uses Buildroot to build an initrd image. The selected configuration disables most of the optional features, with the following notable options enabled:

- Busybox [5],
- Fb-test,
- SDL [30],
- PrBoom [21] (a source port of the Doom engine),
- GCC optimization level set to s.

To make the build reproducible, a new target called `vvsoc_defconfig` has been introduced.

In addition to disabling most optional features, several manual modifications were made to the resulting filesystem to further reduce its final size:

- Unused shared libraries have been removed.
- Some unnecessary scripts and excess files have been removed.

Source

The source code for Buildroot, along with the configuration for VVSoC, can be found on the `vvsoc` branch of the following fork of the Buildroot repository: `https://github.com/MichalBlk/buildroot`.

Doom IWAD

Although Buildroot provides a PrBoom package, a source port of the Doom engine, an IWAD [10] (game data file) is still required to run the actual Doom game. While Buildroot can supply the shareware Doom IWAD, there are alternative solutions that aim to provide a minimal version of the game by:

- Restricting the texture set.
- Shortening or removing sound effects.
- Removing and/or simplifying levels.

- Providing a short custom demo.
- Restricting animations.

There are a few benefits of choosing a restricted IWAD:

- During game startup and level loading, the engine has fewer resources to read and decompress.
- Fewer textures, simplified animations, and maps reduce rendering time, providing higher frame rates.
- A smaller IWAD reduces flash usage and runtime memory requirements.

VVSoC uses the one-level variant of the Doom: Squashware Edition IWAD [7]. The IWAD is manually added to the filesystem generated by Buildroot.

6.4.3 Mimiker

Unlike Linux, which is just an operating system kernel that relies on additional projects such as GNU to form a complete system, Mimiker is a full operating system that provides the kernel, system libraries, and user-space programs.

The build process produces both the kernel image and the target root filesystem. Although such a solution is convenient for the user, this approach currently comes with two main limitations:

- Mimiker does not provide any configuration tools (such as `menuconfig`) to select the included user-space components.
- The kernel does not provide support for loading compressed initrd images.

Given that the default initial RAM disk image is relatively large compared to typical FPGA flash memory capacity, the generated root filesystem has been manually modified. The following changes have been introduced:

- Debug sections for user programs have been deleted.
- Unused libraries have been removed.
- Some other excess files have been deleted.

Chapter 7

Building Images

Before VVSoC can be run in simulation or on an FPGA board, images of the components described in Chapter 6 and the VirtIO firmware must be built. This chapter describes the latest images available in the `images` directory within the VVSoC repository, as well as the steps necessary to build them from source.

7.1 Toolchain

Since VVSoC is a RISC-V-based platform, we rely on cross-compilation to build the software executed on both the application and VirtIO cores, which requires a RISC-V toolchain consisting of two primary components:

- GCC – the GNU C compiler.
- GNU Binutils – a collection of basic programming tools (such as `objcopy`).

A RISC-V toolchain can be obtained as a prebuilt package or built from sources. Most of the images run on VVSoC have been compiled using the toolchain provided by the riscv-gnu-toolchain GitHub repository [8] from the RISC-V Collaboration project. In order to build the toolchain, the following options should be provided during configuration:

- `--with-arch=rv32ima_zicsr_zifencei` – selects the set of supported extensions.
- `--with-abi=ilp32` – enables the 32-bit soft-float variant.

Moreover, since the Linux ELF/glibc variant must be selected, the toolchain should be built using the following command:

```
1 make linux
```

The generated toolchain uses the `riscv32-unknown-linux-gnu-` prefix. Most of the commands listed in this document, as well as the makefiles included in the VVSoC repository, assume that this prefix is used. If a different compatible toolchain is used, the commands and makefiles should be modified accordingly.

7.2 Primary Bootloader

The latest images for the primary bootloader can be found in `images/bootloader`.

The `images/bootloader` directory contains three variants of the primary bootloader, which differ in the size of the initrd image they load:

- `bootloader.bin` – initrd images up to 6 MiB.
- `bootloader_medium.bin` – initrd images up to 16 MiB.
- `bootloader_large.bin` – initrd images up to 48 MiB.

The second and third variants are provided for testing purposes, where the root filesystem includes additional testing programs and scripts.

In addition to the images given above, two bootloader variants are provided for RISC-V testing purposes (11.1):

- `images/bootloader/bootloader_ac_test.bin` – a smaller bootloader that loads tests for the application core.
- `images/bootloader/idle.bin` – an idle loop for testing of the VirtIO core.

7.2.1 Building

To build the images from source, move to the `fw/bootloader` directory and run:

```
1 make
```

7.3 OpenSBI

The VVSoC repository provides a single prebuilt image of OpenSBI at `images/opensbi.bin`.

7.3.1 Building

In order to build the OpenSBI image for VVSoC, follow the steps below:

1. Clone the `vvsoc` branch from `https://github.com/MichalBlk/opensbi`.
2. Move to the cloned repository.
3. Execute:

```
export CROSS_COMPILE=riscv32-unknown-linux-gnu-
export PLATFORM=vvsoc
make
```

The generated images are located at `build/platform/vvsoc/firmware`, in particular the binary image `fw_jump.bin`.

7.4 Device tree

The VVSoC repository provides DTB files in `images/<operating system>`.

To compile a device tree source into a device tree blob, a Device Tree Compiler (DTC) is necessary. On most systems, DTC is available as a standard package through the system package manager.

7.4.1 Linux

For Linux, several variants of the device tree images are provided in `images/linux`. The files differ with respect to the following two choices:

- Whether the system console is exposed through the VirtIO console device or a framebuffer console is used instead.
- The size of the provided initrd image. Similar to the primary bootloader, three size classes are distinguished: standard, medium, and large.

The list of provided device tree images is given below (hvc stands for hypervisor console, and fbc stands for framebuffer console):

- `dt_hvc.dtb`,
- `dt_fbc.dtb`,
- `dt_hvc_medium.dtb`,
- `dt_fbc_medium.dtb`,
- `dt_hvc_large.dtb`,
- `dt_fbc_large.dtb`.

Similar to the primary bootloader, the variants that handle medium and large initrd images are provided for testing purposes.

Building

In order to compile the device tree sources into device tree blobs, move to `fw/dts/linux` and run:

```
make
```

7.4.2 Mimiker

For the purpose of running Mimiker on VVSoC, two device tree variants are provided in `images/mimiker`:

- `dt.dtb`,
- `dt_test.dtb`.

The only difference between the two files is that the latter instructs the system to run all kernel and user-space tests instead of performing the standard operation.

Building

To generate the DTB files, navigate to `fw/dts/mimiker` and execute:

```
make
```

7.5 Linux

In the case of Linux, the kernel and user-space components are built separately.

7.5.1 Kernel

The VVSoC repository provides two variants of the Linux kernel in `images/linux`:

- `kernel.bin`,
- `kernel_test.bin`.

The second image is used to run Linux kernel-space tests, which are described in 11.2.2.

Building

Instructions on how to build the variant with kernel testing enabled are provided in 11.2.2. To build the standard kernel image, perform the following steps:

1. Clone the `https://github.com/MichalBlk/linux-vvsoc` repository.
2. Move to the kernel repository.
3. Execute:

```
export CROSS_COMPILE=riscv32-unknown-linux-gnu-
export ARCH=riscv
make vvsoc_defconfig
make
```

The resulting binary image can be found at `arch/riscv/boot/Image`.

7.5.2 User-Space

The following is a list of initrd images provided in `images/linux`:

- initrd.cpio,
- initrd_test.cpio,
- initrd_test_full.cpio.

The second and third variants are used to run Linux user-space tests, which are described in 11.2.3.

Building

Building initrd images for user-space testing is described in 11.2.3. The steps to generate the standard root filesystem image are given below:

1. Clone the `vvsoc` branch from `https://github.com/MichalBlk/buildroot`.
2. Move to the repository.
3. Execute:

```
make vvsoc_defconfig
make
```

The produced root filesystem image, in the form of a cpio archive (`rootfs.cpio`), is available in the `output/images` directory.

Doom IWAD

To complete the root filesystem, a Doom IWAD is required. Since VVSoC uses the Squashware IWAD version, follow the steps below to obtain the image:

1. Visit `https://github.com/fragglet/squashware/releases/tag/squashware-1.3`.

2. Download and unpack the one-level version `squashware-1lev-1.3.zip`.

Manual Rootfs Modification

The IWAD image must be added to the root filesystem. Moreover, in the final image provided in the VVSoC repository (`images/linux/initrd.cpio`), some unnecessary files have been removed to save space and speed up the boot process. This step is optional, as the image should still fit into flash memory if the step is omitted.

To obtain the final root filesystem image, perform the following actions:

1. Move to `buildroot/output/images`.

2. Unpack the archive:

```
mkdir rootfs
cd rootfs
cpio -idv < ../rootfs.cpio
```

3. Include the Squashware Doom IWAD:

```
cp <PATH TO THE IWAD> usr/share/games/doom/doom1.wad
```

4. Optionally delete unnecessary files. For instance, the `readelf` tool can be used to check which dynamic libraries have been linked against any of the included user applications, and the unused libraries can be removed from the final filesystem. As an example, execute the following command to see the list of shared libraries required by the BusyBox executable:

```
readelf -d bin/busybox | grep 'NEEDED'
```

5. Build the final cpio archive:

```
find . | cpio -o -H newc -R root:root > ../rootfs.cpio
```

7.6 Mimiker

Mimiker is a complete operating system that includes both the kernel and user-space components. For this reason, Mimiker provides a customized toolchain for each supported architecture, which is used to build all the elements of the system.

7.6.1 Toolchain

To build the toolchain, perform the steps below:

1. Clone the `vvsoc` branch from `https://github.com/MichalBlk/mimiker`.
2. Move to `mimiker/toolchain/gnu`.
3. Run:

```
1 make
```

The built toolchain is presented in the form of a package. It is also provided in a raw form in `toolchain/gnu/riscv32-mimiker-elf/debian/tmp/usr/bin`.

7.6.2 Operating System

The VVSoC repository contains a single kernel image (`kernel.bin`) as well as a single initrd image (`initrd.cpio`) for Mimiker in `images/mimiker`.

Building

To build the system, navigate to the main directory of the Mimiker repository and execute:

```
1 make
```

The produced images include:

- `sys/mimiker.img` – binary image of the kernel.
- `initrd.cpio` – the final root filesystem as a cpio archive.

As explained in 6.4.3, the default root filesystem may be too large, and manual modification may be necessary. To minimize the filesystem, perform the steps below:

1. Move to `sysroot`.

2. Remove files containing debug sections for user programs:

```
find . -name "*.dbg" -delete
```

3. Since currently Mimiker does not provide a compiler and all user programs are linked statically, the `lib` and `usr/include` directories can be removed altogether.

```
rm -rf lib usr/include
```

4. Build the final cpio archive:

```
find . | cpio -o -H newc -R root:root > ../initrd.cpio
```

7.7 VirtIO Firmware

Alongside building images used by the application core, the image for the firmware executed by the VirtIO core must be built as well.

The VirtIO firmware is provided as part of the VVSoC repository. The source code can be found in `fw/virtio`, and the compiled image is available in `images`.

7.7.1 Building

To build the image, navigate to `fw/virtio` and run:

```
make
```

Chapter 8

Emulation

One of the major advantages of choosing VirtIO as the interface for peripheral devices is its broad support in UNIX-like operating systems, including Linux, as well as in virtualization tools such as QEMU. As a result, a platform similar to the target system can be easily emulated on a standard PC, which has two main benefits:

- It simplifies software development, with particular benefits for porting new operating systems.
- It enables the user to investigate the intended behavior of existing software components (e.g., the Linux kernel). Furthermore, most emulators allow the user to connect to the emulated system with a debugger.

8.1 QEMU

QEMU (Quick Emulator) is a full-system emulator and virtualizer with the following characteristics:

- QEMU is open-source with a large active community.
- The tool supports many processor architectures (e.g., RISC-V, ARM, x86).
- QEMU provides functional models of many peripheral devices, including VirtIO devices.
- It does not provide cycle-accurate hardware behavior and instead focuses on functional correctness.
- QEMU uses Dynamic Binary Translation [22] (DBT), which means that it performs the following procedure:
    1. Take a basic block of guest instructions.

2. Translate the block into an equivalent sequence of host instructions.
3. Execute the host instructions.
4. Cache the translated block and reuse the sequence the next time the same guest code runs.

This avoids slow instruction-by-instruction interpretation and is the main reason why QEMU is fast.

8.1.1 Obtaining QEMU for RISC-V

The QEMU system emulator for the 32-bit variant of the RISC-V architecture is called `qemu-system-riscv32`. It can be obtained through standard Linux distribution packages (e.g., via the `qemu-system-riscv32` package on Debian).

8.1.2 Emulation

To emulate a system similar to VVSoC running Linux in QEMU, execute the following command:

```
qemu-system-riscv32 -M virt -smp 1 -m 250M \
    -kernel <PATH TO THE KERNEL IMAGE> \
    -initrd <PATH TO THE INITRD IMAGE> \
    -append "earlycon=sbi console=hvc0 root=/dev/ram" \
    -chardev stdio,mux=on,id=chd0 \
    -display sdl -device virtio-serial-device \
    -device virtconsole,chardev=chd0 \
    -serial chardev:chd0 -device virtio-keyboard-device \
    -device virtio-gpu-device
```

The following is a description of the selected options:

- `-M virt` – choose the virtual platform.
- `-smp 1` – emulate only one CPU.
- `-m 250M` – set the size of main memory to 250 MB.
- `-kernel <PATH>` – provide the path to the Linux kernel image (e.g., `images/linux/kernel.bin`).
- `-initrd <PATH>` – provide the path to the initial RAM Disk image (e.g., `images/linux/initrd.cpio`).
- `-append "earlycon=sbi console=hvc0 root=/dev/ram"` – add the given parameters to the kernel command line. The provided string is taken from `fw/dts/linux/dt_hvc.dts`.

- `-display sdl` – use the SDL backend to show the guest's graphical output.
- `-chardev stdio,mux=on,id=chd0` – create a character device associated with stdio with multiplexing enabled. This is necessary as both the VirtIO console and the early console used from firmware will transmit characters to standard output.
- `-device virtio-serial-device` – create a VirtIO-MMIO serial device.
- `-device virtconsole,chardev=chd0` – create a VirtIO-MMIO console as a port of the VirtIO serial device. The console will use the previously created character device.
- `-serial chardev:chd0` – associate the early console used by firmware with the character device.
- `-device virtio-keyboard-device` – create a VirtIO-MMIO keyboard device.
- `-device virtio-gpu-device` – create a VirtIO-MMIO GPU device.

To enable GDB debugging of the emulated system, execute:

```
qemu-system-riscv32 -M virt -smp 1 -m 250M \
    -kernel <PATH TO THE KERNEL IMAGE> \
    -initrd <PATH TO THE INITRD IMAGE> \
    -append "earlycon=sbi console=hvc0 root=/dev/ram" \
    -chardev stdio,mux=on,id=chd0 \
    -display sdl -device virtio-serial-device \
    -device virtconsole,chardev=chd0 \
    -serial chardev:chd0 -device virtio-keyboard-device \
    -device virtio-gpu-device \
    -gdb TCP::<PORT NUMBER> -S
```

The command includes two new options:

- `-gdb TCP::<PORT NUMBER>` – open a gdbserver on the specified TCP port.
- `-S` – stop the execution until a GDB connection is established and the continue command is received.

Chapter 9

Simulation

To simulate a hardware design means to run a software model of that design. The main advantage of simulation is that functional correctness and timing behavior can be checked before the design is physically implemented (e.g., on an FPGA board).

When it comes to functional simulation, we can distinguish two types of simulators: event-driven and cycle-accurate.

Event-driven simulators:

- They simulate the Verilog event scheduling semantics exactly (e.g., delays).
- Every signal change can trigger re-evaluation of dependent processes — even within the same simulation time step.
- Suitable for behavioral, RTL, and gate-level simulation.

Cycle-accurate simulators:

- They assume a synchronous clocked design.
- They do not support arbitrary event scheduling or delays (no intra-cycle scheduling).
- The design is simulated in the following steps:
  1. Evaluate all combinational logic.
  2. Advance all flip-flops at the clock edge.
  3. Repeat for the next cycle.
- Suitable for RTL-level simulation.

In the case of large synchronous designs like CPUs or SoCs, progressing the simulation on a per-clock-cycle basis is much more efficient than simulating every single signal change. For this reason, VVSoC is simulated with a cycle-accurate simulator.

9.1 Verilator

Verilator [35] is a tool that compiles synthesizable Verilog and SystemVerilog designs into high-performance C++ or SystemC models, which are cycle-accurate. There are several advantages to using Verilator for cycle-accurate simulation:

- Verilator is open-source with massive community support.
- It is widely used in academia and industry.
- The generated code is highly optimized.
- Verilator can handle millions of gates efficiently, making it a great choice for large designs.
- Verilator models can be integrated with C++/SystemC software.
- The produced C++ code can be profiled, instrumented, and debugged using standard tools.

Verilator is used to generate a cycle-accurate C++ software model of VVSoC.

9.2 Target Board Definition

As mentioned in Chapters 2 to 4, the implementation of some hardware components depends on the target board on which VVSoC will be implemented. There are three such components:

- Flash controller,
- Main memory controller,
- Keyboard controller.

Each supported target board provides a dedicated directory within the `hw` directory containing:

- An implementation of the three components.
- A board-specific top module (`_top.sv`) that instantiates the board-independent top module (`top.sv`).
- A header file (`board.svh`) that defines the `board` package which contains board-specific parameters.

The following is the list of parameters defined by `board.svh`:

- Board-specific configurable parameters described in Chapters 2 to 4, namely:
  - `FLSZ`,
  - `MMEMSZ`,
  - `MMEM_DATALEN`,
  - `VGA_COLORLEN`.
- The frequency of the system clock in Hz (`CLK_FREQ`).
- The name of the target board (e.g., `NEXYS_A7`). This definition is used within the board-independent code to enable certain board-specific fragments for synthesis (e.g., instantiating the memory controller module may require passing a different set of signals depending on the target board).
- Lengths of board-specific signals passed to the board-independent top module (e.g., the low-level DRAM address length).

Only a single board-specific directory (e.g., `hw/nexys_a7`) should be provided to the synthesis tool.

9.2.1 Target Board for Simulation

For the purposes of simulation, the `sim` target board has been provided, which has the following characteristics:

1. The `sim` board does not include a board-specific top module (`_top.sv`); instead, a testbench is used.
2. Main memory and flash memory are implemented as simple Verilog memory modules.
3. The size of the flash memory is set to 64 MiB.
4. The size of the main memory is set to 256 MiB.
5. Main memory transactions are 256 bits wide.
6. During simulation, images are loaded into flash memory by the simulator.
7. Currently, a keyboard controller is not implemented.
8. Each VGA color is 8 bits wide.
9. The system clock frequency is set to 100 MHz.

9.3 Images for Simulation

Unlike in the case of FPGA target boards, where binary images have to be manually programmed into flash memory, in simulation, all images are loaded into flash directly by the simulator. This direct loading is enabled by a dedicated system task in SystemVerilog called `$readmemh` [9, Sec. 21.4]; however, it requires a specific image format.

A `.mem` file is a plain-text file containing raw hexadecimal data that can be used to initialize a Verilog memory array during both simulation and FPGA configuration.

For a VVSoC simulation to work, the following images must be present in the `sim` directory:

- `bootloader.mem` – primary bootloader image.
- `opensbi.mem` – OpenSBI image.
- `kernel.mem` – operating system kernel image.
- `dt.mem` – device tree blob image.
- `initrd.mem` – initial RAM disk image.
- `virtio.mem` – VirtIO firmware image.

Binary images can be converted using the tools provided in the VVSoC repository.

9.3.1 Converting Binary Images to the `.mem` Format

To convert a binary image into a `.mem` initialization file, use the `bin2mem.py` script located in `/tools`. The script has the following syntax:

```
1 bin2mem.py <PATH TO BINARY> <WIDTH IN BYTES> <PATH TO DESTINATION>
```

The width argument corresponds to the format of the target Verilog memory array and equals the size of a single entry in bytes. In case of generating an image for the primary bootloader or the VirtIO firmware, the argument should be set to four, as this is the word size in bytes for both the application core and the VirtIO core. When converting other images for simulation purposes, the argument should be set to one, since flash reads transfer a byte at a time.

The third argument is optional. If not provided, the generated `.mem` image is placed in the current working directory with the `.mem` extension. For example, converting `dt_hvc.dtb` produces `dt_hvc.mem`.

9.4 Testbenches

A simulation testbench is a piece of code that tests and verifies a hardware design. A testbench performs the following main actions:

1. Instantiate the simulated design, which is referred to as the Design-Under-Test (DUT).
2. Drive the input signals.
3. Monitor and optionally verify the generated outputs.

Since Verilator generates a C++ software model of VVSoC and provides a class representing the design, the VVSoC testbenches are written in C++. All provided testbenches are located in the `sim` directory.

9.4.1 Simulating Peripheral Devices

One of the key aspects of hardware simulation is the modeling of peripheral devices, which entails generating input signals and processing the output signals of the implemented hardware controllers.

UART

VVSoC uses the C++ UART model (`UARTSIM`) provided as part of the wbuart32 project [40] for Verilator simulation. The model is configured through the `setup` member function to match the characteristics of the UART controller implemented in VVSoC (5.3.2), as well as to account for the system clock frequency of 100 MHz.

During simulation, input data for VVSoC is read from standard input, and bytes transmitted by the SoC are printed to standard output.

Keyboard

Currently, neither a keyboard controller for the `sim` target board nor a keyboard model for simulation is provided.

VGA

A simple VGA model has been implemented using the SDL library. The model performs the following steps:

1. For each pixel, the VGA output generated by VVSoC is written to a framebuffer in host memory.

2. Whenever a frame is completed and differs from the previous one, an SDL texture is updated and rendered in a dedicated window.

9.4.2 Standard Testbench

The standard testbench used to simulate VVSoC is `tb.cpp`. The software model for standard simulation is generated with high-performance optimizations enabled.

The standard testbench performs the following notable actions:

1. Assertion of the initial reset.
2. Simulation of the system clock.
3. Simulation of UART (using `UARTSIM`).
4. Simulation of VGA (using SDL).

9.4.3 Debugging Testbench

Although optimization options used for standard simulation significantly improve performance, in the optimized model, only the ports of the top module of the design are exposed as public members of the VVSoC class.

For debugging purposes, the `tb_dbg.cpp` testbench is provided, and the options used during software model generation enable the testbench to access any signal within the entire SoC design.

The debugging testbench performs the same actions as the standard testbench and optionally prints the address of the current instruction executed by the application core to standard error.

Accessing Nested Signals

Verilator-generated classes and their members reflect the hierarchy and signal names of the original HDL design, using an additional prefixing and scoping scheme. For example, the `mstatus` control and status register can be accessed as `dut->__PVT__top->__PVT__APP_CORE->__PVT__CSR_REG_FILE->mstatus_r`.

Although the debugging testbench accesses only a single nested signal, the instruction pointer of the application core, it can be easily modified by the user to examine any other signals in a similar way.

The generated software model includes debugging symbols; therefore, in addition to accessing signals from the testbench, signal values can also be examined from a software debugger such as GDB.

Differences Between Verilator Versions

The prefixing and scoping rules that Verilator applies to represent module hierarchy and avoid name collisions for nested signals are not part of a stable public API and may change between Verilator versions.

All testbenches provided in the VVSoC repository that access nested signals have been developed to work with the 5.006 2023-01-22 revision of Verilator. In case of a version where a different scheme is used, the testbenches should be modified accordingly.

9.4.4 Measuring Testbench

The measuring testbench (`tb_measure.cpp`) performs the same tasks as the standard testbench and provides the following additional functionality:

- For each type of instruction handled by the application core, the testbench counts:
  - The number of times an instruction of this type has been encountered.
  - The total number of cycles spent executing instructions of this type.
- The simulation terminates when the number of instructions executed by the application core reaches the value specified by the `INSTRUCTIONS` parameter defined in the testbench.
- At the end of the simulation, the testbench reports:
  - The counters maintained for each instruction type.
  - The number of instruction cache accesses and instruction cache hits.
  - The number of TLB accesses and TLB hits.
  - The number of main cache accesses and main cache hits.

Since the measuring testbench accesses numerous nested signals, the target software model is generated with the same options as for the debugging testbench.

9.4.5 Generating a Software Model

To build a software model of VVSoC, follow the steps below:

1. Navigate to the `sim` directory.
2. Depending on the target testbench, execute a specific command:
   - For standard simulation run:

```
make
```

- For debugging simulation run:

```
make dbg
```

- For measuring simulation run:

```
make measure
```

The generated model and its corresponding executable image are located in `sim/obj_dir`.

If the user has not provided the `.mem` images, the makefile will also generate the default images for running Linux.

If a software model has already been generated and the user wishes to change the target testbench, the current model must first be deleted by running:

```
make clean
```

9.4.6 Running a Simulation

To run a standard VVSoC simulation, perform the following steps:

1. If the user does not want to use the default `.mem` initialization images produced by running make, appropriate binary images from the `images` directory should be converted using `tools/bin2mem.py` and placed in the `sim` directory.
2. Select a testbench and generate the corresponding software model.
3. Run the generated binary executable (`sim/obj_dir/Vtop`).
4. To end the simulation, click the close button in the SDL window.

Chapter 10

FPGA Implementation

When it comes to implementing a design in actual hardware, there are two main options:

- Application-Specific Integrated Circuit (ASIC) – a custom-manufactured chip.
- Field-Programmable Gate Array (FPGA) – a type of Programmable Logic Device (PLD) capable of implementing complex digital logic.

Choosing an FPGA implementation offers the following benefits:

- FPGAs can be reprogrammed repeatedly, making them ideal for prototyping, teaching, and research.
- ASIC development involves very high fabrication costs, whereas FPGA development typically requires only a development board and design tools, with the tools often available free of charge for academic use.
- FPGAs allow rapid design testing, debugging, and iteration directly in hardware.
- FPGA vendors (like Xilinx/AMD and Altera/Intel) provide extensive IP cores and development tools, reducing implementation effort.
- Many universities already have FPGA development boards, which means that students and researchers can easily access and experiment with real hardware without additional investment.

VVSoC has been designed for FPGA implementation. Currently, the Nexys A7 is the only supported development board, but the design can be ported to other FPGA boards, as described in the following sections.

10.1 Nexys A7

The Nexys A7 is a development board based on the Xilinx Artix-7 FPGA family. It is widely used in academic courses, research, and prototyping due to its combination of affordability, accessibility, and sufficient resources to implement small- to medium-scale digital systems, including custom processors and accelerators.

The Nexys A7 is available in two variants, Nexys A7-50T and Nexys A7-100T, which differ only in FPGA capacity. The tools used to implement VVSoC on the Nexys A7 have been configured to work with the 50T variant of the board. If the 100T variant is used instead, the tools must be adjusted accordingly.

10.2 Target Board Definition

As described in 9.2, each target board must provide a board directory within the `hw` directory that implements the components listed below:

- Flash controller,
- Main memory controller,
- Keyboard controller,
- Board-specific top module (`_top.sv`),
- Header for board-specific parameters (`board.svh`).

10.2.1 Target Board for the Nexys A7

The implementation of board-specific components for the Nexys A7 is contained in the `hw/nexys_a7` directory.

Flash Controller

The Nexys A7 is equipped with a 16 MiB flash device [29] connected through a dedicated Quad-mode SPI (QSPI) bus. The implemented flash controller has the following characteristics:

- As described in 6.1.1, only read operations are currently performed on flash; therefore, write operations are not implemented.
- Although the flash device supports four-bit commands, the controller relies on the traditional single-bit serial input.

- The generated slave clock operates at the frequency of 50 MHz, which is the maximum frequency supported by the single-bit read command.
- Upon power-up, the controller issues the reset command, which restores the device to its initial state.

Main Memory Controller

The Nexys A7 contains a 128 MiB DDR2 SDRAM component. To interface with this memory, VVSoC relies on a low-level memory controller provided by Xilinx. The controller IP is generated using the Memory Interface Generator [1, Sec. 1.2] (MIG) with the following notable options selected:

- AXI interface disabled (a native-FIFO style is used instead).
- PHY clock frequency of 300 MHz.
- 2:1 PHY to controller clock ratio.
- Data width of 16 bits.
- 4 bank machines.
- Strict ordering mode.
- Input clock frequency of 200 MHz.
- Sequential burst type.
- Bank-row-column address mapping.

The generated controller provides a well-defined User Interface (UI) that can be used to access the DRAM. The VVSoC main memory module for the Nexys A7 is a simple state machine that receives requests from the cache and translates them into UI commands sent to the generated controller.

The size of a single data word for the generated user interface is determined as follows:

1. Since the data width is 16 bits and a double rate is used, a single beat transfers 32 bits.
2. Setting the PHY to controller clock ratio to 2:1 means that 2 beats are performed during each UI cycle.
3. Therefore, the size of a single UI data word is equal to 64 bits.

Because Double Data Rate (DDR) full bursts consist of 8 beats, in theory, a total of 256 bits can be transferred in a single full burst, 64 bits at a time. However, in practice, some of the later beats may not be accessible. The main memory module for VVSoC assumes that at least the first four beats are always available through the UI; therefore, the size of a single transfer (`MMEM_DATALEN`) is set to 128 bits.

Keyboard Controller

The Nexys A7 contains a USB Type-A connector and an auxiliary microcontroller that acts as a USB HID host. The microcontroller translates USB HID boot protocol transactions into PS/2 signals, which are then provided to the FPGA. Consequently, the Nexys A7 FPGA chip can interface with any standard USB keyboard using the PS/2 protocol.

The keyboard module for VVSoC is a straightforward PS/2 keyboard controller that receives scancodes and translates them into evdev events, which are subsequently sent to the VirtIO manager.

Board-Specific Top Module

The board-specific top module has two main goals:

- Generate clock signals required by the board-independent top module.
- Instantiate the board-independent top module and connect the appropriate FPGA pins to its ports.

The `_top` module for the Nexys A7 generates the following clock signals:

- System clock (100 MHz).
- Clock for the memory controller (200 MHz).
- Clock for the flash controller (100 MHz).
- Clock for the VGA controller (25 MHz).

The clocks are generated by an instance of the Phase-Locked Loop (PLL) IP [36, Sec. 5.78] provided by Xilinx.

The `_top` module also gains access to the slave clock signal for the SPI flash, which remains a dedicated signal after configuration.

Board-Specific Parameters

For the Nexys A7, `board.svh` defines the following parameters:

- `FLSZ` is set to 16 MiB.
- `MMEMSZ` is set to 128 MiB.
- `MMEM_DATALEN` is set to the size of a half DDR burst (128 bits).
- `VGA_COLORLEN` is set to 4, as this is the number of bits the Nexys A7 VGA port uses to represent each color.
- `CLK_FREQ` is set to 100 MHz.
- In addition, several parameters are defined to specify the lengths of low-level DRAM signals that are passed to the board-specific main memory module by the board-independent top module.

10.3 Implementation Tools

There are two major FPGA vendors: Xilinx/AMD and Altera/Intel. Each vendor provides a proprietary tool used to implement hardware designs on the FPGAs they produce:

- Vivado [38] for Xilinx/AMD FPGAs.
- Quartus Prime [24] for Altera/Intel FPGAs.

The following is a typical list of steps performed to implement a design on an FPGA:

1. A new project is created in the target implementation tool, and the appropriate FPGA model is selected.
2. The design source files are added to the project.
3. Optionally, a subset of IP cores provided by the tool is added to the project.
4. A constraints file is added to the project.
5. The synthesis process is run.
6. The implementation process is run, which includes three main stages:
   (a) Mapping,
   (b) Placement,

(c) Routing.

7. The implemented design is converted to a binary configuration file (bitstream) that can be loaded onto the FPGA.

8. The FPGA is programmed with the generated bitstream.

Each target FPGA board for VVSoC should provide a dedicated directory within `tools/vivado` or `tools/quartus`, depending on the FPGA vendor. The directory should contain project files that enable VVSoC to be implemented on the target FPGA board using the appropriate implementation tool. For each target board, the project name should be set to `vvsoc`.

10.3.1 Nexys A7 Vivado Project

Since the Nexys A7 contains a Xilinx FPGA, Vivado is the tool used for FPGA implementation. The project files for implementing VVSoC on the Nexys A7 are provided in the `tools/vivado/nexys_a7` directory. The following is a list of included files:

- `vvsoc.tcl` – a project recreation script. It is used to build a Vivado project for VVSoC.
- `vvsoc.srcs/sources_1/ip/mig` – a directory containing files that define the memory interface generator IP used by the project.
- `vvsoc.srcs/sources_1/ip/pll` – a directory for the phase-locked loop IP used by the project.
- `nexys_a7_50t.xdc` – a design constraints file. It specifies all physical and timing constraints for the project, including FPGA pin assignments.

With a system clock frequency of 100 MHz and default Vivado project settings, timing closure for VVSoC on the Nexys A7 can be difficult. For this reason, different synthesis and implementation strategies have been selected for the project.

Synthesis Strategy

The synthesis strategy has been set to `Flow_PerfOptimized_high`, which has the following characteristics:

- Increased number and depth of optimization steps.
- Stronger logic restructuring.
- Greater exploration of alternative logic mappings.

- Disabled resource sharing.
- One-hot FSM extraction.

Although the strategy results in an improved worst-case slack and overall timing, it has two main drawbacks:

- Increased synthesis runtime.
- Potentially higher resource utilization.

Implementation Strategy

For implementation, the `Performance_ExplorePostRoutePhysOpt` strategy has been selected, which has two main characteristics:

- It performs timing-driven physical optimizations by exploring alternative placement and routing options.
- The strategy performs additional post-route physical optimizations to reduce timing violations.

Similar to the `Flow_PerfOptimized_high` synthesis strategy,
the `Performance_ExplorePostRoutePhysOpt` implementation strategy can result in increased implementation runtime and higher resource usage.

Generating a Vivado project

To generate a complete Vivado project for implementing VVSoC on the Nexys A7, navigate to the `tools/vivado/nexys_a7` directory and run:

```
1 make
```

The resulting project will be located in `tools/vivado/nexys_a7/vvsoc`.

10.3.2 Flash Images

Before loading the design onto the FPGA, the flash memory on the target board must be programmed with the images that will be loaded by the primary bootloader. Both Vivado and Quartus provide means to program the on-board flash. Although the details differ, the two main steps are as follows:

1. Generate a configuration file that contains all the binary images to be written to the flash, along with a starting address for each image.

2. Program the flash using the configuration file.

Since the only currently supported target FPGA board contains a Xilinx FPGA, only Vivado flash programming is described in detail.

Vivado Flash Configuration Files

In Vivado, flash configuration files are called memory configuration files and use the `.mcs` file extension. Currently, all flash images provided in the VVSoC repository use the quad-mode SPI interface.

The following is a list of flash images provided for Linux in `images/flash/vivado/qspi/linux`:

- `fl_hvc.mcs` – standard image with the system console exposed through the VirtIO console device.
- `fl_fbc.mcs` – standard image that employs the framebuffer console.
- `fl_ktest.mcs` – image for kernel-space testing (11.2.2).
- `fl_utest.mcs` – image for user-space testing (11.2.3).

On the other hand, `images/flash/vivado/qspi/mimiker` contains the following flash images for running Mimiker:

- `fl.mcs` – standard image.
- `fl_test.mcs` – image for kernel- and user-space testing (11.3).

Additionally, the `images` directory contains an image for RISC-V testing of the application core (11.1) `images/flash/vivado/qspi/fl_ac_test.mcs`.

Vivado Flash Configuration File Generation

To generate a memory configuration file for VVSoC, follow the steps below:

1. Open a Vivado project.
2. Open the tool for creating memory configuration files by selecting `Tools > Generate Memory Configuration File...` in Vivado.
3. Set the `Custom Memory Size` field to 16 MB.
4. Set `Interface` depending on the bus used to connect the target flash and FPGA. For a quad-mode SPI flash (including the one on the Nexys A7), select `SPIx4`.

5. Select the `Load data files` option.

6. Add an entry for each of the images listed below. Note that the starting addresses must match the addresses used by the primary bootloader, which are defined in `fw/include/soc.h`. For example, since the flash base address in VVSoC is 0xf0000000 and `FL_KERNEL_START` in `fw/include/soc.h` is defined as 0xf0021000, the kernel image should use a starting address of 0x21000.

   - Binary OpenSBI image (e.g., `images/opensbi.bin`) loaded at 0x000000.
   - Device tree blob (e.g., `images/linux/dt_hvc.dtb`) loaded at 0x020000.
   - Binary kernel image (e.g., `images/linux/kernel.bin`) loaded at 0x021000.
   - Initrd cpio archive (e.g., `images/linux/initrd.cpio`) loaded at 0x421000.

7. Confirm the selected options and generate a memory configuration file by clicking `OK`.

Vivado Flash Programming

With an `.mcs` configuration file in place, the on-board flash can be programmed by performing the following steps:

1. Open a Vivado project.

2. Connect the target FPGA board to the PC.

3. In the `Flow Navigator` pane on the left, from the `PROGRAM AND DEBUG` actions select `Open Hardware Manager`.

4. From the newly available actions, select `Open Target > Auto Connect`.

5. In the `Hardware` pane in the opened hardware manager, click right on the target FPGA (`xc7a50t_0` for the Nexys A7-50T) and select `Add Configuration Memory Device...`

6. In the `Add Configuration Memory Device` window, in the search box, enter the name of the target flash device (`s25fl128sxxxxxx0-spi-x1_x2_x4` for the Nexys A7) and select the search result.

7. Accept the selected flash device by selecting `OK` and when prompted whether you wish to program the device right now, select `OK` again.

8. In the `Program Configuration Memory Device` window, locate an `.mcs` file for VVSoC (e.g., `images/flash/vivado/qspi/linux/fl_hvc.mcs`) in the `Configuration file` field.

9. Select `Apply` and then `OK` to start the programming process.

10.3.3 Bitstreams

Similar to flash images, only Vivado bitstream generation and FPGA programming are explained in detail.

Vivado Bitstreams

Vivado uses the `.bit` file format to store bitstreams. Each target FPGA board for VVSoC should provide the following bitstreams in the `images/<board name>` directory:

- `vvsoc.bit` – standard bitstream.
- `vvsoc_medium.bit` – bitstream for Linux user-space testing (11.2.3).
- `vvsoc_ac_test.bit` – bitstream for RISC-V testing of the application core (11.1).
- `vvsoc_vc_test.bit` – bitstream for RISC-V testing of the VirtIO core (11.1).

10.3.4 Vivado Bitstream Generation

To generate a VVSoC bitstream, follow the steps below:

1. Complement the project files by providing initialization files for block RAMs:

```
./tools/bin2mem.py images/bootloader/bootloader.bin 4 \
    tools/vivado/<board name>/bootloader.mem
./tools/bin2mem.py images/virtio.bin 4 \
    tools/vivado/<board name>/virtio.mem
```

2. Open the project file `vvsoc.xpr` for the target board in Vivado (e.g., `tools/vivado/nexys_a7/vvsoc/vvsoc.xpr`).

3. In the `Flow Navigator` pane on the left, from the `PROGRAM AND DEBUG` actions select `Generate Bitstream`.

4. Vivado will prompt you to launch synthesis and implementation first; select `Yes`.

5. In the `Launch Runs` window select `OK`.

After completing all steps, the resulting bitstream can be found at `tools/vivado/<board name>/vvsoc/vvsoc.runs/impl_1/_top.bit`.

10.3.5 Vivado FPGA Programming

To load the VVSoC bitstream onto the FPGA, follow the steps below:

1. Open a Vivado project.
2. Connect the board to the PC.
3. Open the hardware manager and open a new target as explained in Vivado flash programming (10.3.2).
4. In the `Flow Navigator` pane on the left, from the `PROGRAM AND DEBUG` actions select `Program Device` and choose the target FPGA when prompted (`xc7a50t_0` for the Nexys A7-50T).
5. In the `Program Device` window, use the bitstream search box to locate a `.bit` file (e.g., `images/<board name>/vvsoc.bit`), and then select `Program`.

Chapter 11

Tests

Currently, the following types of tests are available for VVSoC:

- RISC-V tests for the application core.
- RISC-V tests for the VirtIO core.
- Linux kernel-space tests.
- Linux user-space tests.
- Mimiker kernel-space tests.
- Mimiker user-space tests.
- Framebuffer test from a Buildroot package.

Finally, the ultimate practical test for Linux is running Doom. In the case of Mimiker, which does not currently provide a VirtIO GPU driver, running a terminal-based Tetris is considered the final test.

In addition to describing different classes of tests, the last section of the chapter examines the results obtained by running the measuring testbench.

11.1 RISC-V Tests

RISC-V tests for unprivileged extensions verify whether a set of extensions is supported correctly by providing numerous unit tests for different variants of each instruction defined by each extension.

VVSoC uses RISC-V tests adapted from the PicoRV32 project [20], which test the following two extensions:

- Base integer instruction set.

- Integer multiplication and division.

The tests are located in the `fw/tests` directory and are available for both the application core and the VirtIO core; however, for the latter, only the RV32I extension is verified, as the M extension is not supported.

The tests use UART to report the progress of execution.

11.1.1 Building Images

Although the latest images are available at `images/ac_test.bin` and `images/vc_test.bin`, the images can be generated by navigating to `fw/tests` and running:

```
1 make
```

11.1.2 Running Tests on App Core in Simulation

To run the RISC-V tests on the application core in simulation, follow the steps below:

1. Move to the `sim` directory.
2. Prepare images by running:

```
1 make ac_test_imgs
```

3. Follow the standard steps to run a simulation (9.4.6).

The following is a list of images used for application core testing:

- `images/bootloader/bootloader_ac_test.bin` – bootloader for loading app core tests.
- `images/ac_test.bin` – test image.

11.1.3 Running Tests on App Core on Hardware

To test the application core on hardware, perform the following actions:

1. Program the on-board flash (10.3.2) with the `images/flash/vivado/qspi/fl_ac_test.mcs` memory configuration file.
2. Program the FPGA (10.3.5) with `images/nexys_a7/vvsoc_ac_test.bit`.

11.1.4 Running Tests on VirtIO Core in Simulation

To test the VirtIO core in simulation, perform the following steps:

1. Move to the `sim` directory.
2. Prepare images by running:

```
make vc_test_imgs
```

3. Follow the standard steps to run a simulation (9.4.6).

The list of images used for running tests on the VirtIO core is given below:

- `images/bootloader/idle.bin` – idle loop image.
- `images/vc_test.bin` – test image.

11.1.5 Running Tests on VirtIO Core on Hardware

To run the RISC-V tests on the VirtIO core on hardware, program the FPGA (10.3.5) with `images/<board name>/vvsoc_vc_test.bit`.

11.1.6 Results

All RISC-V tests for both application and VirtIO cores pass successfully in simulation and on hardware.

11.2 Linux Tests

In operating-system testing, we can distinguish tests based on where they execute:

- Kernel-space tests – execute in privileged mode as part of the kernel. They have direct access to internal kernel data structures, memory, and privileged functionality.
- User-space tests – execute as regular applications. They test the system through published interfaces, such as system calls and device files.

11.2.1 Preferred System Console

Since Linux provides a driver for the VirtIO GPU device, we can choose between a system console exposed through the VirtIO console device and a framebuffer console. The decision is made by selecting an appropriate device tree blob (as explained in 7.4.1).

For testing purposes, it is recommended to communicate with the system through a command-line interface—either via stdio when running in simulation or from a PC connected to the target board when running on hardware. Selecting the console device provides the following benefits:

- The VirtIO console device is the most straightforward way to interact with the system and does not rely on complex kernel subsystems such as the direct rendering manager.
- The user can easily review the messages produced during the testing process.
- Testing output can be copied to a file and subsequently processed with standard text-processing tools.

All flash configuration files provided in `images` for running tests on Linux use a device tree blob that instructs the kernel to use the VirtIO console device for the system console.

11.2.2 Kernel-Space Tests

VVSoC relies on the Linux Kernel Unit Testing [12] (KUnit) framework to provide kernel-space tests.

Building Images

The latest Linux kernel image with kernel unit testing enabled is available at `images/linux/kernel_test.bin`. To build the image from source, follow the standard steps of building the kernel, but instead of using `vvsoc_defconfig` for configuration, use `vvsoc_test_defconfig`.

The `vvsoc_test_defconfig` configuration target contains the following two notable changes:

- Enable kernel unit testing.
- Include all KUnit tests with satisfied dependencies.

How the Tests Start

The KUnit tests are started automatically after the kernel has initialized enough subsystems to execute the tests, before the first user process is started.

Running Tests in Simulation

To run Linux kernel-space tests in simulation, perform the following steps:

1. Move to the `sim` directory.
2. Prepare images by running:

```
make linux_ktest_imgs
```

3. Follow the standard steps to run a simulation (9.4.6).

The standard set of Linux images is used to run kernel-space tests, with one modification: instead of the standard kernel image, a variant that includes unit tests is used.

Running Tests on Hardware

As in simulation, the standard kernel image needs to be replaced with the image that contains the kernel-space tests. To achieve this, program the on-board flash (10.3.2) with `images/linux/fl_ktest.mcs`. Subsequently, follow the standard steps to program the FPGA (10.3.5).

Results

All enabled kernel unit tests pass successfully both in simulation and on hardware.

11.2.3 User-Space Tests

Currently, user-space tests for VVSoC are provided by the Linux Kernel Selftests [14] (Kselftests) framework, which is included within the kernel repository. The framework is meant to verify that specific kernel subsystems behave correctly using small and reproducible tests.

Although Kselftests are relatively lightweight and easy to integrate into the target root filesystem, they are primarily intended as subsystem-level regression tests for kernel developers rather than a complete system test suite. They do not include comprehensive tests of complex cross-subsystem behavior, such as the full process life cycle. For broader coverage and more thorough user-space-level validation, the Linux Test Project [16] (LTP) should be deployed in the future.

Covered Subsystems

To enable a subsystem to be tested on VVSoC, the following requirements must be satisfied:

- Kernel support for the subsystem must be enabled.
- The target root filesystem must include required libraries (e.g., libcap).
- The target root filesystem must provide required tools (e.g., a Python interpreter).

Currently, tests for the following subsystems have been enabled for VVSoC:

- core – essential kernel functionality.
- exec – program execution mechanics.
- kcmp – kernel-level resource comparator.
- proc – `/proc` virtual filesystem.
- RISC-V – architecture-specific components.
- sigaltstack – alternative signal stack support.
- size – size-related constraints and limits.
- tmpfs – temporary in-memory filesystem.
- TTY – terminal interface layer.

Other systems have been disabled due to the lack of at least one of the three requirements listed above.

Source Modifications

Kselftests are provided as part of the Linux kernel repository in `tools/testing/selftests`. Two main changes have been introduced to the source code:

- The list of tested subsystems has been manually selected in the selftests makefile.
- From the set of enabled subsystems, some tests have been disabled by modifying source files due to unsatisfied requirements. Every test that passes on a corresponding QEMU virtual machine described in 8.1.2 has been included for VVSoC testing.

Available Images

The user-space tests are provided in the form of user programs and scripts; therefore, they need to be added to the target root filesystem. The VVSoC repository provides two initrd images for user-space testing:

- `images/linux/initrd_test.cpio`,
- `images/linux/initrd_test_full.cpio`.

The full image contains tests for all the subsystems listed above. However, the resulting filesystem is close to 50 MB in size, which may be too large for a target FPGA board. Therefore, for testing on development boards with lower memory capacity, `initrd_test.cpio` has been provided. The alternative image does not include the exec subsystem, saving around 40 MB.

Toolchain

To build the selftests from source, a toolchain compatible with the target root filesystem is required. A suitable toolchain can be obtained by following the steps for generating the Buildroot root filesystem for VVSoC, as described in 7.5.2. After the build process is finished, the toolchain will be available at `buildroot/output/host/bin`.

Building Images

To build the full rootfs image, follow the steps below:

1. Clone the `https://github.com/MichalBlk/linux-vvsoc` repository.
2. Move to the repository.
3. Compile the selftests and generate a tarball containing the compiled binaries by executing:

```
export CROSS_COMPILE=riscv32-buildroot-linux-gnu-
export ARCH=riscv
make -C tools/testing/selftests gen_tar
```

   The final image will be available at `kselftest_install/kselftest-packages/kselftest.tar.gz` within the selftests directory.

4. Perform the following two modifications to the standard root filesystem image as explained in 'manual rootfs modification' in 7.5.2:

(a) Extract the selftests to `/opt/kselftests` within the target filesystem.

(b) Remove the directories for the disabled subsystem in `/opt/kselftests` to save space.

To create the smaller image, modify the makefile for selftests to disable the exec subsystem.

How the Tests Start

The selftests have to be started manually by running the `/opt/kselftests/run_selftest.sh` script from the system console.

Running Tests in Simulation

To run Linux user-space tests in simulation, follow the steps below:

1. Move to the `sim` directory.
2. Prepare images by running:

```
make linux_utest_full_imgs
```

3. Follow the standard steps to run a simulation (9.4.6).

The following modifications are made to the standard set of images for running Linux:

- Use the primary bootloader image that is adjusted for loading a large initrd image (`images/bootloader/bootloader_large.bin`).
- Change the device tree blob to reflect the increase in the initrd size (`images/linux/dt_hvc_large.dtb`).
- Substitute the standard rootfs image with the image containing Kselftests (`images/linux/initrd_tests_full.cpio`).

Running Tests on Hardware

To run the selftests on hardware, perform the two steps below:

1. Program the flash device (10.3.2) with `images/flash/vivado/qspi/linux/fl_utest.mcs`, which includes the rootfs image with a limited number of selftests, as well as the appropriate device tree image.

2. Program the FPGA (10.3.5) with `images/<board name>/vvsoc_medium.bit`, which initiates the boot memory with the image of a primary bootloader (`images/bootloader_medium.bin`) capable of loading medium-size initrd archives.

Results

All except one of the enabled Kselftests pass successfully both in simulation and on hardware. The failing test is called `proc: proc-multiple-procfs`.

11.3 Mimiker Tests

Both kernel-space and user-space tests are provided as part of the Mimiker repository and can be run on VVSoC.

11.3.1 Building Images

Kernel-space tests are built and included as part of the kernel by default; therefore, the standard kernel image (`images/mimiker/kernel.bin`) can be used to run the tests.

Similar to kernel-space tests, user-space tests are also built by default during operating system compilation. The tests are included in the root filesystem in the form of a single user program `bin/utest`. For these reasons, the standard initrd image (`images/mimiker/initrd.cpio`) can be used to run the tests.

11.3.2 How the Tests Start

To run all kernel-space and user-space tests, the `test=all` parameter assignment must be included in the kernel command line, which is specified in the device tree source. The appropriate device tree blob is available at `images/mimiker/dt_test.dtb`.

11.3.3 Running Tests in Simulation

To run Mimiker tests in simulation, follow the steps below:

1. Move to the `sim` directory.
2. Prepare images by running:

```
1 make mimiker_test_imgs
```

3. Follow the standard steps to run a simulation (9.4.6).

One modification is made to the standard set of images used for running Mimiker: instead of the standard DTB, the device tree description that enables kernel and user tests is used.

11.3.4 Running Tests on Hardware

In order to run the tests on hardware, use
the `images/flash/vivado/qspi/mimiker/fl_test.mcs` configuration file to program the on-board flash (10.3.2). Next, program the FPGA by performing the standard steps (10.3.5).

11.3.5 Results

All kernel-space and user-space tests pass successfully both in simulation and on hardware.

11.4 Linux Framebuffer Test

Buildroot provides a framebuffer testing package (`fb-test`). It is a collection of simple utilities that render graphics directly to a Linux framebuffer, which helps to validate that framebuffer support is working correctly.

11.4.1 Running a Test

The `fb-test` package is included in the standard root filesystem image for Linux. A simple test can be run by executing `fb-test` in the system console.

The pattern printed by the application during simulation can be seen in Figure 11.1. Note that the image generated when running on hardware may differ slightly from the one presented in the figure due to having fewer bits per color.

11.5 Can It Run Doom?

The main goal of VVSoC is to provide a straightforward design of a general-purpose computer that is not only capable of running UNIX-like operating systems, but also sophisticated enough to run complex applications such as Doom, which is currently the most advanced program run on VVSoC. For this reason, running Doom can be considered the final practical test.

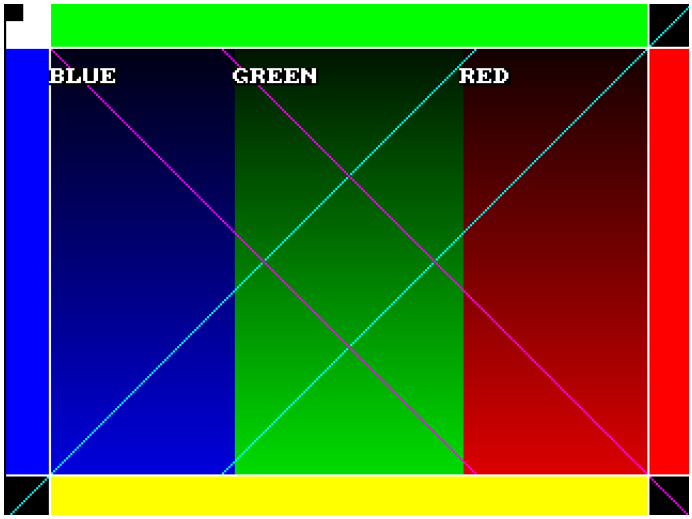

Figure 11.1: `fb-test` run in simulation

Figure 11.2: Doom run in simulation

11.5.1 Running Doom

As explained in 6.4.2, a source port of the Doom engine (PrBoom) along with a restricted IWAD (Doom: Squashware Edition) is provided in the standard root filesystem image for Linux. To run Doom, execute the following command from the system console:

```
1 /usr/games/prboom -width 320 -height 240
```

A screenshot from the initial demo run in simulation is provided in Figure 11.2. Similarly to the framebuffer test, on hardware, the generated image may not exactly match the figure because of fewer bits per color.

Since neither a keyboard controller nor a keyboard model is currently provided

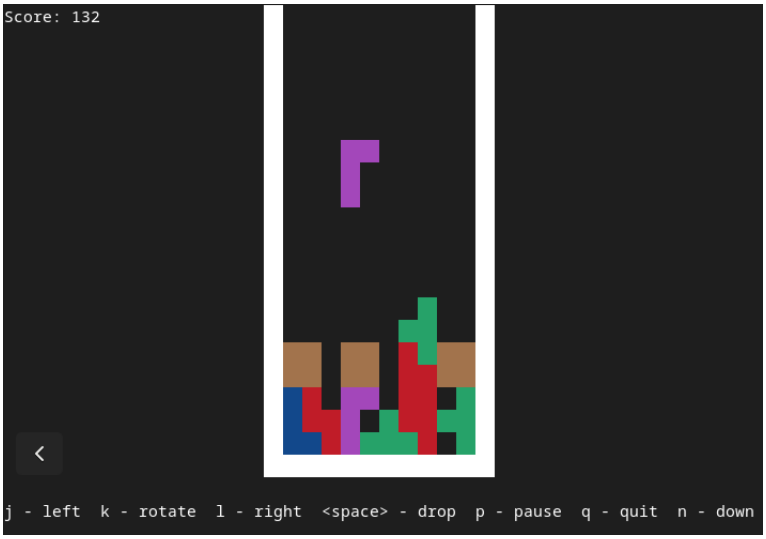

Figure 11.3: Tetris run on hardware

for simulation, Doom has to be run on hardware to enable an interactive experience where the game can actually be played.

11.6 Tetris on Mimiker

Currently, a port of the terminal-based Tetris from NetBSD is one of the most advanced non-graphical applications supported by Mimiker.

11.6.1 Running Tetris

Tetris is included by default in the initrd image for Mimiker. To run Tetris, simply execute `tetris` from the system console.

A screenshot of a Tetris game run on hardware is provided in Figure 11.3.

11.7 Measuring Testbench Results

The discussion of VVSoC tests concludes with an examination of the results produced by running the measuring testbench in cycle-accurate simulation with the following settings:

- The `INSTRUCTIONS` parameter was set to 3×10^8, which means that the simulation was terminated after the application core had executed 3×10^8 instructions.
- The standard set of Linux images was used:

Type	#instructions	#cycles
LUI	2458871	15851090
AUIPC	2749742	14974224
JMP	8485208	39442826
BRANCH	39306661	190936705
LOAD	62490227	587074874
STORE	35828705	327434784
IMM	84472085	424999790
OP	45384529	209007022
MUL	15783688	126825782
DIV	295194	4662961
LR	43889	563211
SC	113922	703362
RMW	233758	3396641
CSR	1776235	10933185
ECALL	73023	365585
EBREAK	0	0
MRET	6773	27092
SRET	70244	280976
SFENCE_VMA	35957	143837
NOP	390553	2369405

Table 11.1: Application core instruction counters

  - `images/bootloader/bootloader.bin`,
  - `images/opensbi.bin`,
  - `images/linux/kernel.bin`,
  - `images/linux/dt_hvc.dtb`,
  - `images/linux/initrd.cpio`,
  - `images/virtio.bin`.

- During the simulation, the system ran numerous standard applications: BusyBox programs, the framebuffer test, and Doom. These applications were launched manually, without any automated scripts. In the future, a robust benchmarking framework should be employed.

For each instruction type handled by the application core, Table 11.1 presents the number of times an instruction of this type has been encountered and the total number of cycles spent executing instructions of this type.

Table 11.2 provides the statistics reported for the instruction cache, translation lookaside buffer, and the main cache.

Component	#accesses	#hits	#misses
ICACHE	236274022	203170753 (86%)	33103269 (14%)
TLB	109421822	104432480 (95%)	4989342 (5%)
CACHE	137650388	115742975 (84%)	21907413 (16%)

Table 11.2: ICACHE, TLB, and CACHE statistics

Chapter 12

Summary

Whereas most open-source general-purpose computers are sophisticated systems with codebases reaching tens of thousands of lines, VVSoC is a straightforward alternative that trades performance for simplicity. It utilizes VirtIO-MMIO as the target interface for peripheral devices and avoids hardware complexity by implementing the interface in software on a simple dedicated processor. VVSoC provides an educational example of a complete general-purpose computer targeted at an FPGA board.

12.1 Contributions

The following is a list of notable contributions that I have made as part of my work:

- I designed an application core with the following characteristics:
  - Multi-cycle processor.
  - 32-bit little-endian RISC-V architecture.
  - Supported ISA extensions: RV32I, M, A, Zifencei, and Zicsr.
  - Supported privilege modes: M, S, and U.
  - Contains an MMU with hardware page walks.
  - Contains a set-associative TLB with ASID support.
  - Contains an alternative direct-mapped TLB.
  - Contains a simple, virtually indexed, and virtually tagged instruction cache.
- I designed a subsystem called the application subsystem with components that support the operation of the application core, including:
  - CLINT,

  - Boot memory,
  - Debug console.

- I successfully employed the VirtIO-MMIO protocol as the target interface for peripheral devices. The protocol is implemented in software by a dedicated core, which limits the hardware implementation to a set of memory-mapped registers per VirtIO device. I implemented the following VirtIO devices:
  - VirtIO console device,
  - VirtIO keyboard device,
  - VirtIO GPU device.

- I designed a simple dedicated core for the VirtIO protocol (called VirtIO core) with the following characteristics:
  - Multi-cycle processor.
  - 32-bit little-endian RISC-V architecture.
  - The only supported ISA extension is RV32I.
  - The only supported privilege mode is M.

- I designed a subsystem called the VirtIO subsystem with components that support the operation of the VirtIO core:
  - VirtIO memory,
  - VirtIO manager.

- I designed the following board-independent controllers for peripheral devices:
  - UART,
  - VGA.

- I designed a physically indexed and physically tagged set-associative cache that is shared between both processors.

- I provided an implementation of a target board for simulation, including:
  - Flash controller,
  - Main memory module.

- I provided an implementation of a target board for the Nexys A7 FPGA board, including:
  - Flash controller,
  - Main memory module,
  - Keyboard controller,
  - Vivado project files.

- I wrote a primary bootloader.
- I provided a port of OpenSBI for VVSoC.
- I adjusted the Linux kernel to work on VVSoC.
- I provided a Buildroot filesystem for Linux with Busybox, a framebuffer testing application, Doom engine source port, and a reduced Doom IWAD.
- I adjusted the Mimiker operating system to work on VVSoC, which included fixing existing kernel bugs and writing a VirtIO console driver.
- I wrote a firmware that runs on the VirtIO core (VirtIO firmware).
- I provided device tree sources for both Linux and Mimiker.
- I provided a QEMU emulation of a similar system.
- I wrote several Verilator testbenches.
- I adjusted the RISC-V tests from PicoRV32 to work on VVSoC for testing both cores.
- I provided a variant of the Linux kernel with kernel unit tests enabled.
- I adjusted Linux kernel selftests to work on VVSoC.
- I provided a variant of Mimiker image with both kernel- and user-space tests enabled.
- I provided a number of memory configuration files for Vivado flash programming.
- I provided several bitstreams of VVSoC for the Nexys A7.

12.2 Results

The main results of my work are given below:

- VVSoC is a complete general-purpose computer with support for console, keyboard, and display.
- A similar system can be easily emulated in QEMU.
- The system can be simulated with cycle-accurate Verilator.
- The system can be implemented on the Nexys A7 FPGA board.
- VVSoC is advanced enough to run Doom on Linux. Moreover, since the system contains a keyboard and a display, the game can actually be played.

- VVSoC can run the Mimiker operating system. This is the first time that Mimiker has successfully been run on a real RISC-V hardware platform.
- The system passes all but one of the provided tests, including:
  - RISC-V tests for both processors,
  - Linux kernel-space tests,
  - Linux user-space tests (here is the failing test),
  - Mimiker kernel-space tests,
  - Mimiker user-space tests,
  - Framebuffer test from a Buildroot package.

12.3 Related Work

The main inspiration for VVSoC was RVSoC [18], which is a RISC-V SoC project that introduced me to the idea of using VirtIO-MMIO as the target interface for peripheral devices and providing a simple dedicated processor to implement the VirtIO protocol in software. RVSoC was developed by Junya Miura, Hiromu Miyazaki, and Kenji Kise at the Tokyo Institute of Technology.

12.4 Future Work

Although a lot has been accomplished, there is still considerable room to grow. The following is a list of possible future improvements and enhancements:

- The system could be ported to enable implementation on other FPGA boards.
- VVSoC could become even more functional by providing support for other VirtIO devices, such as a mouse, a disk, and a network interface.
- The simple VirtIO processor could be sped up by introducing pipelining.
- The processor word size should become a configurable parameter.
- VirtIO shared memory regions could be used to speed up the VirtIO GPU device.
- The RISC-V cores should be thoroughly verified with respect to both standard and privileged extensions.
- The Linux test project should be deployed for Linux user-space testing.
- The system should be evaluated in terms of performance using a set of well-established and representative benchmarks.

- A GDB stub should be implemented to facilitate debugging.
- Alternative replacement policies for instruction cache, TLB, and main cache could be examined.
- The system could be enhanced to provide a symmetric multiprocessing (SMP) variant with multiple application cores.

Additionally, effort should be directed toward increasing the performance of the application core, which is a non-trivial task. The most natural approach would be to introduce pipelining; however, since the processor implements all three privilege modes and associated CSR registers, this can significantly increase both the complexity and code size of the design, conflicting with the primary aim of providing a straightforward general-purpose computer. For this reason, alternative solutions that preserve the simplicity of the project should be considered.

12.5 Closing Remarks

Designing a general-purpose computer is a wonderful, long, and challenging journey. It requires significant knowledge in the following areas:

- Computer architecture,
- Digital logic,
- Target processor ISA,
- Embedded systems,
- Simulation,
- FPGA development.

Despite the challenges, the process is very rewarding and provides invaluable insight into how computers work. It is my hope that this project will serve as a resource for those seeking to deepen their understanding of this field.

Bibliography

[1] *7 Series FPGAs Memory Interface Solutions User Guide*. UG586. Version 4.2. AMD. Nov. 13, 2024. URL: https://docs.amd.com/r/en-US/ug586_7Series_MIS.

[2] Fabrice Bellard. "QEMU, a fast and portable dynamic translator". In: *Proceedings of the Annual Conference on USENIX Annual Technical Conference*. ATEC '05. Anaheim, CA: USENIX Association, 2005, p. 41.

[3] Michał Błaszczyk. "Port of Mimiker Operating System for RISC-V Architecture". Bachelor's thesis. University of Wrocław, Feb. 14, 2022.

[4] *Buildroot web page*. URL: https://buildroot.org.

[5] *BusyBox web page*. URL: https://busybox.net.

[6] *Devicetree Specification*. Version 0.4. The Devicetree Project. June 28, 2023. URL: https://github.com/devicetree-org/devicetree-specification/releases/tag/v0.4.

[7] *DOOM: Squashware Edition repository*. URL: https://github.com/fragglet/squashware.

[8] *GNU toolchain for RISC-V repository*. URL: https://github.com/riscv-collab/riscv-gnu-toolchain.

[9] "IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language". In: *IEEE Std 1800-2023 (Revision of IEEE Std 1800-2017)* (2024), pp. 1–1354. DOI: 10.1109/IEEESTD.2024.10458102.

[10] *IWAD Doom Wiki*. URL: https://doomwiki.org/wiki/IWAD.

[11] Florent Kermarrec et al. "LiteX: an open-source SoC builder and library based on Migen Python DSL". In: *CoRR* abs/2005.02506 (2020). arXiv: 2005.02506. URL: https://arxiv.org/abs/2005.02506.

[12] *KUnit - Linux Kernel Unit Testing*. URL: https://docs.kernel.org/dev-tools/kunit/index.html.

[13] *Linux Input Documentation*. URL: https://www.kernel.org/doc/html/latest/input/input.html.

[14] *Linux Kernel Selftests*. URL: https://docs.kernel.org/dev-tools/kselftest.html.

[15] *Linux repository*. URL: https://github.com/torvalds/linux.

[16] *Linux Test Project repository*. URL: https://github.com/linux-test-project/ltp.

[17] *Mimiker repository*. URL: https://github.com/cahirwpz/mimiker.

[18] Junya Miura, Hiromu Miyazaki, and Kenji Kise. "A portable and Linux capable RISC-V computer system in Verilog HDL". In: *CoRR* abs/2002.03576 (2020). arXiv: 2002.03576. URL: https://arxiv.org/abs/2002.03576.

[19] *OpenSBI repository*. URL: https://github.com/riscv-software-src/opensbi.

[20] *PicoRV32 repository*. URL: https://github.com/YosysHQ/picorv32.

[21] *PrBoom web page*. URL: https://prboom.sourceforge.net.

[22] Mark Probst. "Dynamic Binary Translation". In: *Proceedings of the UKUUG Linux Developers' Conference 2002*. Bristol, United Kingdom, 2003. URL: https://api.semanticscholar.org/CorpusID:16335388.

[23] *Quartus Prime Pro Edition User Guide Platform Designer*. Version 25.1.1. Intel. 2025. URL: https://docs.altera.com/r/docs/683609/25.1.1/quartus-prime-pro-edition-user-guide-platform-designer/answers-to-top-faqs.

[24] *Quartus web page*. URL: https://www.altera.com/products/development-tools/quartus.

[25] *RISC-V Advanced Core Local Interruptor (ACLINT) Specification*. Version 1.0-rc4. RISC-V International, Platform Specification Task Group. Jan. 10, 2022. URL: https://github.com/riscvarchive/riscv-aclint/releases/tag/v1.0-rc4.

[26] *RISC-V Hart-Level Interrupt Controller (HLIC)*. URL: https://www.kernel.org/doc/Documentation/devicetree/bindings/interrupt-controller/riscv%2Ccpu-intc.txt.

[27] *RISC-V Platform-Level Interrupt Controller (PLIC) Specification*. Version 1.0.0. RISC-V International. Mar. 2023. URL: https://github.com/riscv/riscv-plic-spec/releases/tag/1.0.0.

[28] *RISC-V Supervisor Binary Interface (SBI) Specification*. Version 3.0. RISC-V International, Platform Runtime Services Task Group. July 16, 2025. URL: https://github.com/riscv-non-isa/riscv-sbi-doc/releases/tag/v3.0.

[29] *S25FL128S, S25FL256S 128 Mb/256 Mb FL-S Flash SPI Multi-I/O, 3.0 V Datasheet*. 001-98283 Rev. *T. Infineon Technologies AG. Apr. 21, 2025. URL: https://www.infineon.com/assets/row/public/documents/10/49/infineon-s25fl128s-s25fl256s-128-mb-16-mb-256-mb-32-mb-fl-s-flash-spi-multi-io-3-v-datasheet-en.pdf.

[30] *SDL web page*. URL: https://www.libsdl.org.

[31] *SiFive FU540-C000 Manual*. Version 1p5. Chapter 9: Core-Local Interruptor (CLINT). SiFive, Inc. May 7, 2025. URL: https://www.sifive.com/document-file/freedom-u540-c000-manual.

[32] *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. Version 20191213. RISC-V International. Dec. 13, 2019.

[33] *The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*. Version 20211203. RISC-V International. Dec. 3, 2021.

[34] Michael S. Tsirkin and Cornelia Huck, eds. *Virtual I/O Device (VIRTIO) Version 1.3. OASIS Committee Specification Draft 01*. Latest stage: https://docs.oasis-open.org/virtio/virtio/v1.3/virtio-v1.3.html. OASIS. Oct. 6, 2023. URL: https://docs.oasis-open.org/virtio/virtio/v1.3/csd01/virtio-v1.3-csd01.html.

[35] *Verilator web page*. URL: https://www.veripool.org/verilator.

[36] *Vivado Design Suite 7 Series FPGA and Zynq-7000 SoC Libraries Guide*. UG953. Version 2025.2. AMD. Dec. 17, 2025. URL: https://docs.amd.com/r/en%E2%80%91US/ug953%E2%80%91vivado%E2%80%917series%E2%80%91libraries.

[37] *Vivado Design Suite User Guide: Designing IP Subsystems Using IP Integrator (UG994)*. AMD. 2025. URL: https://docs.amd.com/r/en-US/ug994-vivado-ip-subsystems.

[38] *Vivado web page*. URL: https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html.

[39] *VVSoC repository*. URL: https://github.com/MichalBlk/vvsoc.

[40] *wbuart32 repository*. URL: https://github.com/ZipCPU/wbuart32.